Cooperative Graph Neural Network for Materials

Dipannoy Das Gupta*, Teerachote Pakornchote†, Ashley S. Dale‡,
Qi Zhang*, and Christopher Sutton†

*Department of Computer Science, University of South Carolina, Columbia, SC
Email: dg47@email.sc.edu, QZ5@cse.sc.edu

†Department of Chemistry and Biochemistry, University of South Carolina, Columbia, SC
Email: PAKORNCT@mailbox.sc.edu, CS113@mailbox.sc.edu

‡Department of Materials Science and Engineering, University of Toronto, Toronto, Ontario
Email: ashley.dale@utoronto.ca

***Abstract*—Graph Neural Networks (GNNs) provide benchmark accuracy for the prediction of material properties. In a graph representation of materials, each atomic site is a node, and an edge corresponds to bonds between atoms (defined as two sites lying within a specified cutoff distance). Typically, every node connects to all its neighbors within a pre-defined cutoff, leading to chemically irrelevant long-range contacts and contributing to over-smoothing during message passing. In this work, we propose a cooperative extension for GNNs that augments the traditional graph convolution loop with a learnable edge scoring network. This network assigns a soft edge weight to each bond, which is later passed through the sparsity regularizer during the training to guide the network for pruning low-information edges. This adaptive message passing reduces bandgap prediction error by up to 2.5% on JARVIS and 2.8% on the Materials Project relative to the dense baselines while demonstrating that bonds in the 3–4 Å range are essential for the bandgap, indicating its dependency beyond local environments. For the highly localized property of formation energy, the model retains only $62\% - 68\%$ of the total graph edges while achieving a similar or improved predictive performance (0.062 eV/atom on JARVIS and 0.037 eV/atom on Materials Project) compared to the dense baselines, demonstrating a significant fraction of the structural edges in the crystal graph are chemically redundant for property prediction. Beyond the accuracy, this learned edge-pruning scheme maintains a greater representational disparity across the graph nodes compared to the dense baseline, indicating that our approach mitigates over-smoothing.**

I. Introduction

Graph Neural Networks (GNNs) are powerful tools for a wide range of applications that involve complex relational data [1]. In materials discovery applications, graph-based representations provide a natural framework for describing crystalline materials and have been used in numerous machine learning tasks, such as high-throughput material screening and crystal property predictions. [2], [3] Graph-based representations are well suited for learning structural features directly from the atomic positions because in the graph-based representation, nodes correspond to atomic sites within a structure and edges are "bonds" between neighboring sites.

The graph representation can be used to encode information of the structural topology of a crystal for a given machine learning task. For example, it can provide a set of descriptors which can be used with a standard regressor (kernel regression) and lead to highly accurate predictions [4] indicating the importance of topology in the crystal property prediction. However, these topology only descriptors are insensitive to the atomic geometry changes if the graph connectivity remains unchanged.

Materials graph neural networks therefore commonly use graph convolutional to aggregate information from neighboring nodes based on the connectivity such as the Crystal Graph Convolutional Neural Network (CGCNN) [5], which uses a shared convolution weight matrix for all neighbors, and therefore, cannot distinguish the interaction strength of different neighbors. An important development is from geometry-dependent message-passing approaches (i.e., Message Passing Neural Networks (MPNNs)), in which the interaction between neighboring nodes is explicitly parameterized by geometric quantities such as interatomic distances and bond angles. [6]–[20].

Furthermore, the high capacity of MPNNs and large atomistic datasets such as the OC20 [21], OC22 [22], OC25 [23], OMOL [24], OpenDAC [25], and OMAT [26] has enabled the training of foundation machine-learned interatomic potentials (MLIPs) such as UMA [27], MACE-MP0 [28], and Orb [29]. The foundation models improve the performance of downstream prediction tasks which lead the model to achieve the state of the art accuracy [30], [31].

However, a key aspect that determines the accuracy of MPNNs, is the definition of the connectivity between atoms of the crystal graph. Standard crystal graph generation approaches rely on defining the number of edges between sites based on either a radial bond graph (which connects all atoms within a fixed-cutoff distance between neighbors) or a k-nearest-neighbor graph (which connect a fixed/maximum number k of nearest atoms). Radial cutoffs to define local atomic environments usually range from range from 4 Å to 6 Å [9], [32]–[35]. Moreover, because each node receives information from its neighbors, repeated message passing iterations can propagate information beyond the nearest-neighbor environment. Thus, each additional MPNN layer increases the effective range over which information (i.e., messages) can

be communicated through the graph from distant nodes. To model long range interactions in a given system, larger cutoff values can be used in an MPNN, but this has a drawback that it leads to denser, highly connected graphs.

This density introduces structural noises by aggregating chemically irrelevant long-range information. Specifically, these noises refer to the unimportant edges which are created because of falling into the radial cut-off without any meaningful physical or chemical interaction. This phenomenon leads to sub-optimal performance and also degrades the generalization ability [36], [37]. Therefore, graphs generated from larger unit cells of crystals (e.g., an average of 194 atoms in the Materials Project and 52 in JARVIS), certain atomic interactions may be redundant for message passing. In addition to increased redundancy, a node can aggregate information from the furthest nodes by using higher number of message passing layers; starting from the first hop, an atom with n direct neighbors receives n messages, as the layer increases the number of messages can reach up to the $n \times n \times \cdots = n^k$ for k layers, yielding an exponentially growing receptive field containing $\mathcal{O}(n^k)$ messages. However, the neighborhood aggregation step compresses this massive volume of information into a fixed-sized node embedding, possibly leading to information loss known as over-squashing [38]. Furthermore, in the dense topology, the convolution across higher number of layers can make node-features indistinguishable. This creates an over-smoothing issue [39] where the models lose the unique identity of the graph nodes, affecting the final predictive accuracy.

This motivates the study of more expressive architectures that work on the dense and complex crystal structure to filter out unimportant connections.

Several recent efforts have focused on dynamic graph sparsification, and graph edge pruning methods can play a pivotal role in the context of fine-tuning graph neural networks for downstream tasks [40]–[42]. The reduced number of connections via pruning could speed up the pretraining, fine-tuning, and inference while the representation learning and predictive performance can be preserved [43]–[46]. Approaches have been developed to rewire the graph by connecting relevant nodes and shortening propagation distances [47], [48], or designing new message-passing architectures that act on distant nodes directly, e.g., using shortest-path distances [49], [50], [51], [45], [46], [52]. However, this kind of dynamic topological pruning into the end-to-end GNN can sever the computational graph, leading to gradient instability that prevents the model from learning physically accurate topologies [53], [54]. Furthermore, dynamic subgraph extraction can create CPU-GPU synchronization bottlenecks and memory allocation overheads, resulting in slower execution despite processing fewer edges in the crystal graph [55], [56]. Therefore, a scalable method that can extract a physically accurate topology without compromising predictive accuracy and hardware performance still remains unsolved for materials.

In this work, we propose a Cooperative Graph Neural Network (CoGNN) shown in Figure 1 that integrates dynamic topological sparsification with high-throughput material property prediction. CoGNN augments the traditional graph neural network with an action network that can understand the local geometry and chemistry of a crystal and accordingly assign continuous survival probabilities to every structural edge. Our framework employs a differentiable multiplicative masking in place of non-differentiable physical deletion. This preserves the gradient flow, allowing the network to learn about the bonds responsible for the underlying physics while penalizing topological noises via a sparsity regularizer. We evaluate four variations of the action network, each with a different sparsification approach, on two tasks (prediction of the formation energy and band gap energy) from the JARVIS‘ [57] and Materials Project [58] databases. The results indicate that the action networks can guide the subsequent GNN to match or exceed the predictive accuracy of the dense baselines while substantially reducing the number of edges for message passing which consequently mitigates the over-smoothing.

Furthermore, we address a critical question of the models' generalization capability, i.e., if the edge pruning performed by the networks completely depends on the particular task (property) or comes from the inherent chemistry. Our comprehensive generalization experiments demonstrate that our topological pruning policy can transfer knowledge from one task to another effectively.

II. Results

To evaluate the impact of graph pruning on property prediction, we compare the all four variants(which differ in edge scoring approach) of CoGNN framework against a modified baseline derived from the ALIGNN architecture [9]. The baseline model in this work is the sub-module of the full ALIGNN [9] architecture, in which only the bond graph convolution has been used (i.e., we omitted the usual line-graph convolution layers responsible for three-body interactions in the full ALIGNN). The choice was made to restrict the analysis to the bond distances and node features extracted from the bond graph because all four action networks operate on different combinations of raw bond lengths, edge embeddings, and node embeddings to generate a soft weight for each connection. Thus, the pruning strategy of the action net is essentially learning the optimal bond interactions only from interatomic distances and atom features in order to reduce redundancy or unnecessary edges (i.e., bond interactions) in the crystal graph without deteriorating the model accuracy.

We assess these models on two target properties: the band gap energy (E_g) and the formation energy (E_f). These properties are extracted from the Materials Project (MP) [58] and JARVIS databases [57]. As shown in Figure 5, the JARVIS database has fewer atoms per unit cell on average than the MP database, so that these two databases provide different testing regimes for the CoGNN model. The choice to exclude the line-graph convolutional layers has little effect on ALIGNN's performance. The full ALIGNN model including bond graph convolution and line graph convolution has an MAE of 0.596 ± 0.059 eV and 0.464 ± 0.014 eV for the JARVIS E_g and MP E_g datasets,

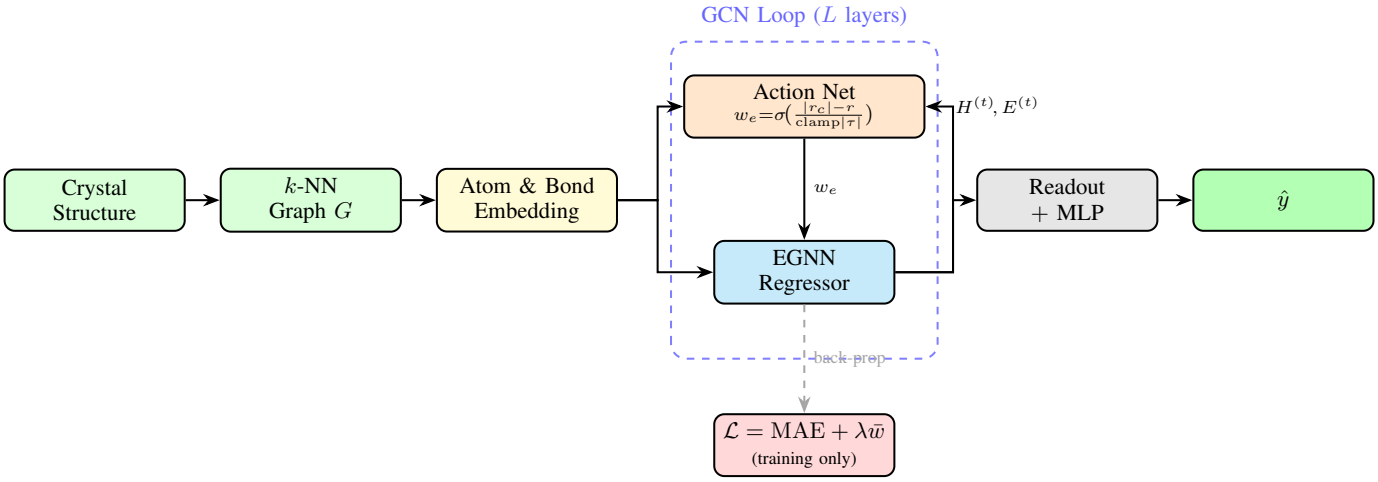


Fig. 1: CoGNN pipeline. A crystal structure is converted to a k-NN bond graph; atoms and bonds are embedded. Within each of L GCN layers the *Action Net* (LEARNABLECUTOFFSCORINGNET by default) assigns a soft weight w_e to every bond(edge); the EGNN regressor uses these weights as edge-gate multipliers. After L layers a graph-mean readout feeds an MLP yielding $\hat{y}$. Training minimises $\mathcal{L} = \text{MAE} + \lambda\bar{w}$ (Eq. 9); at training and inference a hard threshold θ prunes bonds(edges) with $w_e < \theta$.

respectively. In comparison, the ALIGNN model used as our baseline has an MAE of $0.633{\pm}0.047$ eV and $0.493{\pm}0.017$ eV for the JARVIS E_g and MP E_g datasets, respectively. Thus, the average difference in MAE between the ALIGNN model used here and the full ALIGNN model is ≈ 0.030 eV. Similarly, for the formation energy E_f, the full ALIGNN model does not significantly improve the accuracy of the JARVIS and MP: 0.064 ± 0.006 eV/atom and 0.035 ± 0.004 eV/atom , respectively, which are almost identical to the baseline model errors of 0.065 ± 0.006 eV/atom and 0.036 ± 0.006 eV/atom.

All comparisons between CoGNN and ALIGNN have been reported with identical bond graph convolution stacks. The CoGNN model additionally includes an action net which works as an edge scoring network, whereas the baseline ALIGNN model omits the action net and resultant pruning.

Four action network variants are evaluated for the CoGNN model: EDGEONLYSCORINGNET, MULTIPLICATIVEGATESCORINGNET, EDGESCORINGNET, and LEARNABLECUTOFFSCORINGNET. Details of the action network variants are reported in § IV-A. The effect of these variants on the CoGNN model is evaluated using the mean absolute error (MAE) on the test set, and the average edge retention rate (AERR) across all samples in the test set. Because too few edges causes a loss of information that leads to increased prediction error, the optimal action network variant simultaneously reduces MAE and AERR.

A. Band Gap (E_g) Prediction

Tables I and II compare four CoGNN scoring variants implemented against the baseline model when predicting the band gap energy for two benchmark datasets, JARVIS [57] and MP [58]. For each action network variant we report the best threshold θ (formally defined in V-B) over $\{0.1, 0.2, 0.3, 0.4, 0.5\}$ based on the test MAE with a sparsity weight of $\lambda = 0.001$.

TABLE I: Band gap (E_g) test set MAE on JARVIS (5-fold CV, eV). AERR refers to Average Edge Retention rate; lower is better.

Action Net	Best θ	MAE (eV)↓	AERR↓
Baseline (no action)	–	0.633 ± 0.047	100%
EDGEONLYSCORINGNET	0.1	0.658 ± 0.052	$51.34\% \pm 8.68\%$
MULTIPLICATIVEGATESCORINGNET	0.1	0.683 ± 0.046	$45.19\% \pm 5.88\%$
EDGESCORINGNET	0.1	0.658 ± 0.060	$36.76\% \pm 5.15\%$
LEARNABLECUTOFFSCORINGNET	0.3	**0.617** ± 0.053	$93.94\% \pm 0.86\%$

TABLE II: Band gap (E_g) test set MAE on Materials Project (5-fold CV, eV). AERR refers to Average Edge Retention rate; lower is better.

Action Net	Best θ	MAE (eV)	AERR↓
Baseline (no action)	–	0.493 ± 0.017	100%
EDGEONLYSCORINGNET	0.1	0.520 ± 0.019	$45.35\% \pm 7.29\%$
MULTIPLICATIVEGATESCORINGNET	0.1	0.514 ± 0.020	$41.97\% \pm 6.22\%$
EDGESCORINGNET	0.1	0.516 ± 0.014	$41.05\% \pm 3.82\%$
LEARNABLECUTOFFSCORINGNET	0.2	**0.479** ± 0.015	$97.47\% \pm 1.23\%$

For all four CoGNN variants, test set errors for E_g are within ± 0.050 eV of the baseline model. The LEARNABLECUTOFFSCORINGNET consistently achieves the lowest test MAE relative to the baseline model (2.53% and 2.84% lower for the JARVIS and the MP dataset, respectively). The other three variants (EDGEONLYSCORINGNET, MULTIPLICATIVEGATESCORINGNET, and EDGESCORINGNET) exhibit slightly higher errors than the baseline.

Interestingly, for the EDGEONLYSCORINGNET, MULTIPLICATIVEGATESCORINGNET, and EDGESCORINGNET, performances similar to the baseline model are obtained despite the AERR ranging from 41–45% for MP (36% to 51% for JARVIS) for these models. This indicates that a substantial number of edges can be trimmed in a controllable way from the graph without dramatically affecting the model performance. In contrast, the LEARNABLECUTOFFSCORINGNET

prunes only 6% and 3% of edges in the the JARVIS and MP datasets, respectively, indicating that this action network selectively prunes edges that reduce the prediction error.

To gain insight into the decision-making of the action network, we analyzed the edges selected for pruning. Figure 2 visualizes the edge-pruning behavior of the best action network LEARNABLECUTOFFSCORINGNET for each of the four tasks (JARVIS E_g/E_f and MP E_g/E_f). Each row in Figure 2shows the test-set edges for one task (panel a,b,e,f for the E_g), and shows a clear relationship between bond distance and edge pruning for LEARNABLECUTOFFSCORINGNET. The clear relationship can be identified both from the scatter plot (panel a,e) and the histogram (panel b,f) where as the bond length increases, the percentage of edges with the lower weights also increases (panel a,e) and so the dropping occurs exclusively in the higher bond-lengths region (panel b,f). This behavior occurs because the model is explicitly designed to learn from bond lengths (see the IV-A3 for details), starting from an initial bond-length cutoff of 3.5 Å . This value is treated as a learnable parameter and updated throughout training. As the training progresses, the evolving initial bond-length guides the network to assign appropriate weights to the connections(edges) of the atoms. These weights are passed through the θ check where the weights below θ are discarded. The final θ is 0.3 and 0.2 for the JARVIS and MP datasets, respectively, indicating the transition point shifts with the chemistry of each dataset. In contrast, the other action nets, EDGEONLYSCORINGNET, MULTIPLICATIVEGATESCORINGNET, and EDGESCORINGNET have no clear correlation between which edges are trimmed and the bond distances (see SI Figures 1, 2, 3) indicating that edge pruning is rather random relative to bond distances, which leads to slightly higher error compared to the LEARNABLECUTOFFSCORINGNET. The least correlation between the bond distances and edge pruning can be validated with the low JSD score where 0.2472,0.1457,0.0459(0.1625,0.1160,0.0359) is reported for the EDGEONLYSCORINGNET, EDGESCORINGNET, MULTIPLICATIVEGATESCORINGNET using JARVIS(MP) dataset. On the contrary, LEARNABLECUTOFFSCORINGNET shows 0.8399, 0.920 for the JARVIS, MP respectively.

Nevertheless, all four action networks have a common tendency of keeping a higher percentage of bonds with a bond length of 3–4 Å. This suggests that the prediction of E_g requires information beyond nearest neighbor interactions.

To evaluate to what extent that the absence of edges will affect the results, we have trained a model that does not allow for message passing between the edges, which would represent an AERR value of 0%. With no edge convolutions (just node-level) the model has MAE of 1.357 ± 0.085 eV and 0.953 ± 0.038 eV for JARVIS and MP E_g. When all edges are removed from message passing, the atoms can no longer access information about their neighboring environment, which in turn affects the final property prediction. This substantial increase in error highlights the importance of atom–atom interactions for E_g prediction. However, this does not necessarily imply that all interatomic connections must be preserved to achieve the best performance which is the motivation for developing an ideal strategy for pruning only the unimportant edges.

In general, the results for LEARNABLECUTOFFSCORINGNET demonstrate that a selective number of edges can improve model accuracy, which is rationalized to occur because the interatomic distances are important indicators of bond relevance for various electronic properties such as the band gap energy.

B. Formation Energy (E_f) Prediction

We have also applied the same scheme to the formation energies (E_f) of JARVIS and MP; these results are reported in Tables III and IV. Like models trained to predict E_g, the LEARNABLECUTOFFSCORINGNET model has the highest accuracy (MAE = 0.0621 eV/atom) for the JARVIS dataset, which is 4.46% lower than the baseline model (0.065 eV/atom). For the MP dataset, the LEARNABLECUTOFFSCORINGNET model has a similar accuracy (0.037 eV/atom) as the baseline model with only 62% of edges.

However, in contrast to the E_g result where 93% of edges are retained, the E_f percentage is 68% for JARVIS and 62% for MP. This suggests E_f predictions depend significantly on the local environment of each atom while indicating some redundancy in the edges. The LEARNABLECUTOFFSCORINGNET for E_f in Figure 2 (panel c,d,g,h) shows a weaker relationship between interatomic distances and the edges being pruned by the action net compared to what was seen for E_g. This weaker correlation is quantitatively reflected in the lower JSD scores for E_f (0.3065 for JARVIS and 0.3515 for MP) relative to the scores achieved for E_g (0.8399 for JARVIS and 0.9207 for MP).

The scatter plots for E_f in Figure 2 (panel c, g) show a broader transition zone from 2–4 Å in which the model simultaneously retains and prunes edges. In Figure 2(panel d, h), the overlapping region between kept and dropped histograms in the 2–4 Å also supports this behavior of the model. This indicates a model can be sensitive to the local chemical identity alongside the interatomic bonds; bonds of identical length can vary in chemical context depending on the atom types. Conversely, we recall that the scatter plots for E_g in Figure 2 (a, e) display a sharp transition zone indicating raw bond lengths alone serve as a strong predictor in the edge selection for the E_g prediction.

For the other three action nets, a similar fraction of edges are removed; AERR values range from 63–70% for JARVIS and 48–55% for MP. This is in contrast with the action nets trained on E_g, which exhibited much larger variations in the AERR. This consistent level of edge reduction leads to a similar MAE for all action nets, ranging from 0.062–0.073 eV/atom (0.037–0.046 eV/atom) for JARVIS (MP), indicating that high accuracy can be maintained regardless of the specific action network used to determine which edges are pruned. We note that despite 30–40% (45–62%) of the edges being trimmed for JARVIS (MP) these action nets excluding the

MULTIPLICATIVEGATESCORINGNET are performing selective pruning (see SI figure 1,2,3). To demonstrate this point, we tested the accuracy of the model for two scenarios where first one removes all the edges and second one removes different percentage (50%, 80%, 90%) of edges randomly. In the first case, the MAE increases by approx. a factor of 4 (MAE = 0.241 ± 0.029 eV/atom) for the JARVIS dataset and a factor of 5 (MAE = 0.179 ± 0.006 eV/atom) for the MP dataset. These results indicate that the selective pruning of edges minimally affects the MAE, whereas the random removal of the edges also increases error. 50% random pruning leads to MAE = 0.0696 ± 0.006 eV/atom, 0.043 ± 0.004 eV/atom for the JARVIS, MP dataset, respectively.

The distance-based pruning scenario for the JARVIS E_f task observed from the LEARNABLECUTOFFSCORINGNET action network is consistent with the EDGEONLYSCORINGNET results (see SI Figures 1). EDGESCORINGNET shows some degree of similarity as LEARNABLECUTOFFSCORINGNET and EDGEONLYSCORINGNETin the pruning behavior; however, only a few short bonds are dropped for the EDGESCORINGNET (SI Figures 2). EDGEONLYSCORINGNET also shows better separation in the kept and dropped edges for the MP E_f task, while the kept and dropped spectrum overlap at the majority part for the EDGESCORINGNET indicating a weaker correlation (SI Figures 1, 2) (JSD Score 0.178 and 0.086 for the EDGEONLYSCORINGNET and EDGESCORINGNET, respectively). The good correlation between bond-length and edge weight from the EDGEONLYSCORINGNET also reflects in the final accuracy. For example, the EDGEONLYSCORINGNET yields an MAE of 0.0623 eV/atom for the JARVIS dataset and MAE of 0.037 eV/atom for the MP dataset. The MULTIPLICATIVEGATESCORINGNET action network (SI Figure 3) shows the least correlation for both JARVIS E_f and MP E_f (JSD score 0.0298 and 0.015 for JARVIS and MP, respectively). The superimposition of kept and dropped distributions indicates the model's struggle to learn meaningful information regarding bonds. As shown in Table III, the random pruning by MULTIPLICATIVEGATESCORINGNET leads to higher error compared to other three action networks.

TABLE III: Formation energy (E_f) test MAE on JARVIS (5-fold CV, eV/atom).

Action Net	Best θ	MAE (eV/atom)↓	AERR↓
Baseline (no action)	-	0.065 ± 0.006	100%
EDGEONLYSCORINGNET	0.1	0.0623 ± 0.006	$68.07\% \pm 4.22\%$
MULTIPLICATIVEGATESCORINGNET	0.1	0.073 ± 0.012	$70.82\% \pm 4.74\%$
EDGESCORINGNET	0.2	0.066 ± 0.008	$63.52\% \pm 3.04\%$
LEARNABLECUTOFFSCORINGNET	0.5	**0.0621** ± 0.007	$68.36\% \pm 1.46\%$

TABLE IV: Formation energy (E_f) test MAE on Materials Project (5-fold CV, eV/atom).

Action Net	Best θ	MAE (eV/atom)↓	AERR↓
Baseline (no action)	–	**0.036** ± 0.004	100%
EDGEONLYSCORINGNET	0.1	0.037 ± 0.005	$50.78\% \pm 6.07\%$
MULTIPLICATIVEGATESCORINGNET	0.2	0.046 ± 0.003	$48.54\% \pm 7.49\%$
EDGESCORINGNET	0.1	0.038 ± 0.004	$55.51\% \pm 9.17\%$
LEARNABLECUTOFFSCORINGNET	0.5	0.037 ± 0.003	$62.31\% \pm 1.26\%$

C. Effects of Over-smoothing and Over-squashing on pruned graphs

We now investigate how our action network can help to mitigate the effect of over-smoothing and over-squashing in GNNs.

1) MAD-based over-smoothing Analysis: Over-smoothing refers to the node embeddings getting almost similar as the message passing continues for the higher number of graph convolution layers. To analyze the over-smoothing, the Mean Average Distance (MAD) [59] between the representations of the nodes has been used, details discussion in V-E4. MAD can be divided into two parts: local MAD and remote MAD. Local MAD measures the difference of directly connected nodes while the remote MAD calculates the distance of nodes connected via other nodes. The MADGap is calculated according to the equation (14), and the ordering of local MAD and remote MAD in this equation follows the convention established by [59].

Figure 3 shows the evolution of MAE(panel a,d) and MADGap(panel b,e) with the increase of number of GCN layers($L = 1, \ldots, 8$) for the dense baseline and CoGNN on the band gap (E_g) and formation energy (E_f) tasks. For E_g (panel a), test MAE for both models drops sharply between $L = 1$ and $L = 3$ and then flattens around 0.46 eV; the two error curves show similar MAE across every depth with overlapping fold error bars throughout. The corresponding MADGap (panel b) is negative for both models at all depths. This is the result of more dissimilarity observed in the local node representations compared to the remote nodes(see SI figure 5), reflecting the higher chemical heterogeneity of bonded atom pairs compared to the non-bonded pairs in a crystal graph. Notably, CoGNN's MADGap is consistently higher (less negative) than the baseline's at every single depth tested. While a less negative MADGap alone is not the definitive proof of less over-smoothing - since a gap approaching to 0 indicates the complete collapse of local and remote node representations. However, the MAD_{all} for CoGNN persistently remains higher at the higher depths unlike baseline(see SI figure 6). This indicates that the learned edge pruning keeps bonded and non-bonded atom representations more distinguishable than the dense baseline, i.e., less over-smoothing, without any degradation in the predictive performance.

For E_f (panels d, e), the same qualitative pattern holds: test MAE decreases with depth for both models; here CoGNN shows a modest improvement in the MAE as the network depth increases. This small improvement can be attributed to the more local characteristics of formation energy, where pruning is advantageous. CoGNN's MADGap remains above the baseline's at every depth from $L = 1$ to $L = 8$, indicating some degree of over-smoothing mitigation. Unlike E_g, the MAD_{all} does not dominate the dense baseline at the higher depths(see SI figure 6). The MAD_{all} curves for both of the models show a downward progression, indicating the features are losing diversity as the depth increases.

The MADGap analysis shows that CoGNN can maintain

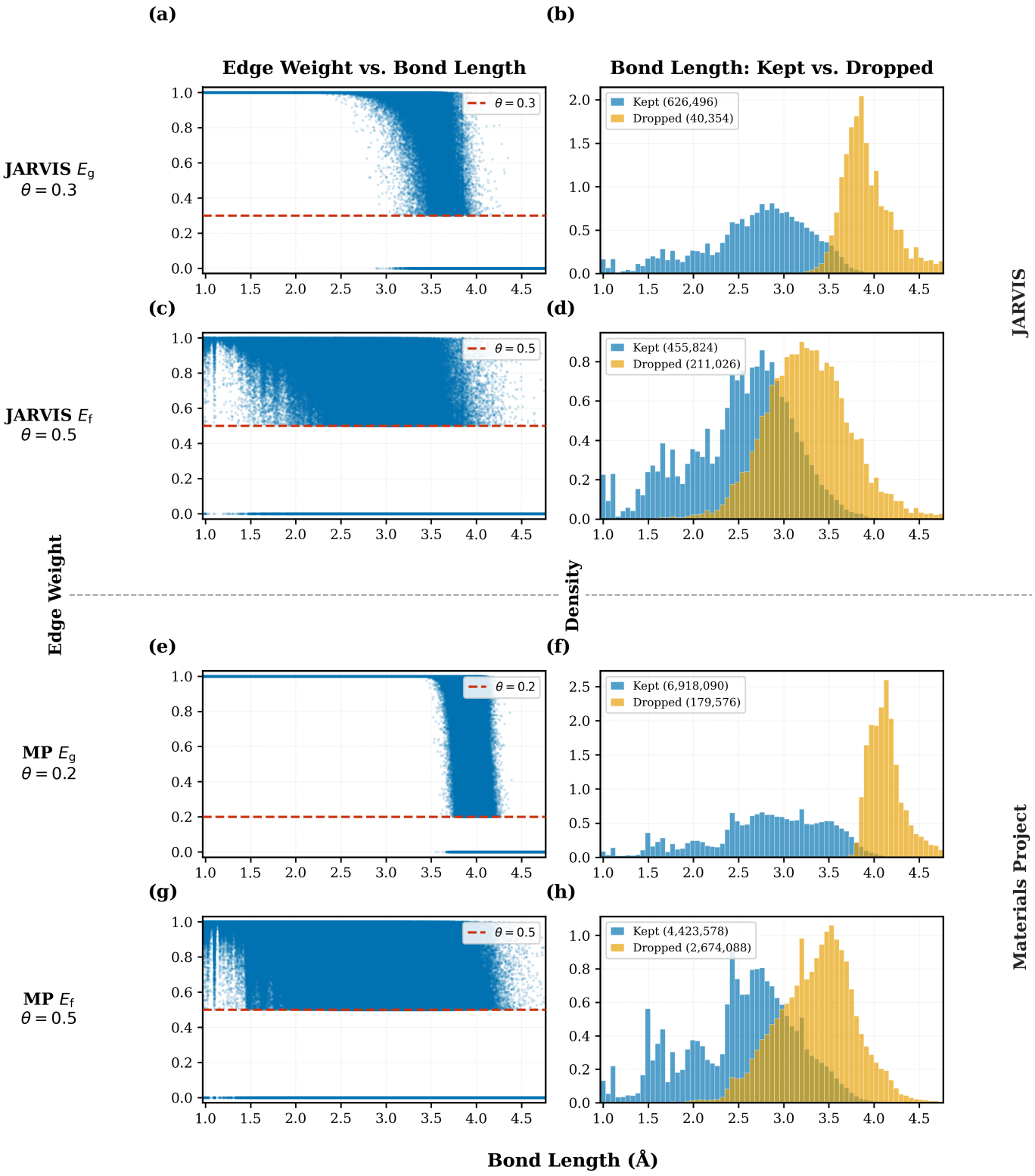


Fig. 2: Edge-pruning analysis for the best action network LEARNABLECUTOFFSCORINGNET on each of the four tasks. **Left column:** Soft edge weight w_e vs. bond length d for all test-set edges. The red dashed line is the edge cut-off threshold θ (JARVIS E_g: θ=0.3; JARVIS E_f: θ=0.5; MP E_g: θ=0.2; MP E_f: θ=0.5); edges below the line are discarded at training and inference time. **Right column:** Bond-length density for retained edges ($w_e \geq \theta$, blue) and pruned edges ($w_e < \theta$, amber). Across all tasks the model learns a near-monotone distance gate: short bonds are retained, while long bonds are pruned, confirming that the learned cutoff recovers a physically meaningful distance-based bond criterion.

the higher disparity in the node representations for the E_g task while this rich variation of features doesn't elevate the predictive performance. Though the E_f task achieves a higher MADGap than baseline, the MADs across all nodes degrade at a similar rate for both of the models. In both cases, the minimal impact on the MAE aligns with the claim that the heterogeneous graph-like crystal structures are more resilient to the increasing network depth [60], [61].

2) Rank-Based Oversmoothing Analysis: The MADGap can not serve as a conclusive, standalone indicator of over-smoothing as it yields negative values at every depth for both of the models. These kinds of distance-based metrics, including Dirichlet-energy, MAD could not capture over-smoothing for the shallow network(e.g. 1-8 layers) while the predictive performance gets degraded [60]. Therefore, we have measured the over-smoothing via the *rank* of the node representation matrix at the final GCN layer, using two continuous relaxations of matrix rank – the numerical rank ($\mathrm{NumRank}$) and the effective (Shannon) rank (Erank) [60], defined formally in Section V-E5. Both metrices reaches to 1, when the representations collapse to single direction(full over-smoothing) and grow toward $\min(N, H)$(discussed in V-E5) when the representations have more independent directions of variation. As a result, higher value at a given depth indicates less over-smoothing.

Figure 3(panel c,f) reports $\mathrm{NumRank}$ and Erank, computed on the final-layer node embeddings and averaged over the 5 cross-validation folds, as a function of network depth ($L \in \{1, \ldots, 8\}$ GCN layers) for both models on the band gap (E_g) and formation energy (E_f) tasks. The two metrics show contrasting perspectives on the representational space. $\mathrm{NumRank}$ shows no consistent separation between the baseline and CoGNN; the two curves intersect repeatedly at nearly every depth for both properties, with overlapping fold error bars, indicating that the two models maintain a similarly dominant leading representation direction. Erank, in contrast, shows a clear and depth-growing advantage for CoGNN at essentially every depth for both properties. For E_f, the baseline's effective rank remains flat and low ($\mathrm{Erank} \approx 12$–$14$) across all eight depths, while CoGNN's climbs from $\mathrm{Erank} \approx 17$ at $L = 1$ to $\mathrm{Erank} \approx 37$ at $L = 8$, with non-overlapping error bars from $L \approx 3$ onward. At $L = 8$ specifically, CoGNN attains $\mathrm{Erank} = 36.92 \pm 2.51$ against a baseline of 14.28 ± 0.85, while $\mathrm{NumRank}$ is statistically indistinguishable between the two models at the same depth (1.822 ± 0.086 vs. 1.855 ± 0.071). The same qualitative pattern holds for E_g, though with a noisier baseline curve at the greatest depths.

Unlike $\mathrm{NumRank}$ which is primarily sensitive to the single strongest direction, Erank is sensitive to the richness of the entire singular-value spectrum. Consequently, these results indicates that, CoGNN preserves substantially more structure in the remaining, secondary directions of variation than the dense baseline while both models retain a similarly strong leading representational direction as depth increases. Combined with the comparable test MAE across all depths (Figure 3,panel a, d), this supports the interpretation that the learned edge pruning mitigates over-smoothing without sacrificing predictive accuracy. This validates the central claim of this work that comparable accuracy is achievable from substantially less structural information.

3) Over-squashing via Jacobian Sensitivity: For analyzing over-squashing [62], we have calculated the Jacobian sensitivity [38], [62] for each atom in the crystal sitting at the different positions(details discussed in V-E3). Jacobian sensitivity $S(d)$ indicates how the perturbations of a node change the final prediction. A model that can preserve long-range interaction will show a flat or slightly decaying average sensitivity $\bar{S}(d)$ curve as distance d between graph nodes increases. Figure 4 reports $\bar{S}(d)$ for the baseline (dense) and CoGNN (pruned) models at three depths ($L \in \{1, 5, 8\}$) on the Materials Project band gap (E_g) and formation energy (E_f) tasks. The baseline and CoGNN curves are superimposed at most of the graph distances except some of the higher distances at $5, 6$. Also no significant decay is found as the graph distance increases. At the higher distances, for the higher number of GCN layers baseline shows better sensitivity than CoGNN. This means pruning edges might have removed some of the message-passing paths without which the information is not getting propagated to the farthest nodes. For the E_g the difference is higher than the formation energy at $d = 5$ as formation energy is more of a local property. So, limiting the information flow is not hurting the formation energy E_f much as the bandgap E_g at the higher number of GCN layers.

D. Cross Property Generalizability

An important question in the general application of this strategy towards a pre-trained task is whether the learned edge-pruning strategy overfits to the specific property on which it was trained. To investigate whether the model only learns the topology for a given property or whether the resulting graph can be applied towards other tasks, we tested the transferability of one graph topology optimized for E_f (Phase 1) which was subsequently used to predict E_g (Phase 2). We repeated this experiment for the LEARNABLECUTOFFSCORINGNET on both the MP and JARVIS datasets. The results summarized in table V for the LEARNABLECUTOFFSCORINGNET with the optimal pruning thresholds selected for each property (discussed above), E_f and E_g ($\theta = 0.5$(MP); $\theta = 0.2, 0.3$ (JARVIS), respectively). The training and evaluation from phase 1 provides optimal sparsified graphs, which are extracted and kept fixed for phase 2. In phase 2, a standard baseline model (without an action network) was trained from scratch to predict a different target property than the source property using only the pruned graphs generated from phase 1. When the graph topology was optimized for MP E_f (Phase 1) and subsequently used to predict MP E_g (Phase 2), the model achieved a test MAE of 0.511 ± 0.013 eV. This represents only a marginal degradation compared to the dense-graph baseline (MAE = 0.493 eV) and the optimized graph for that specific property (MAE = 0.479 ± 0.015 eV). We recall that the graph for E_f retained only 68% and 62% of the edges for the JARVIS and

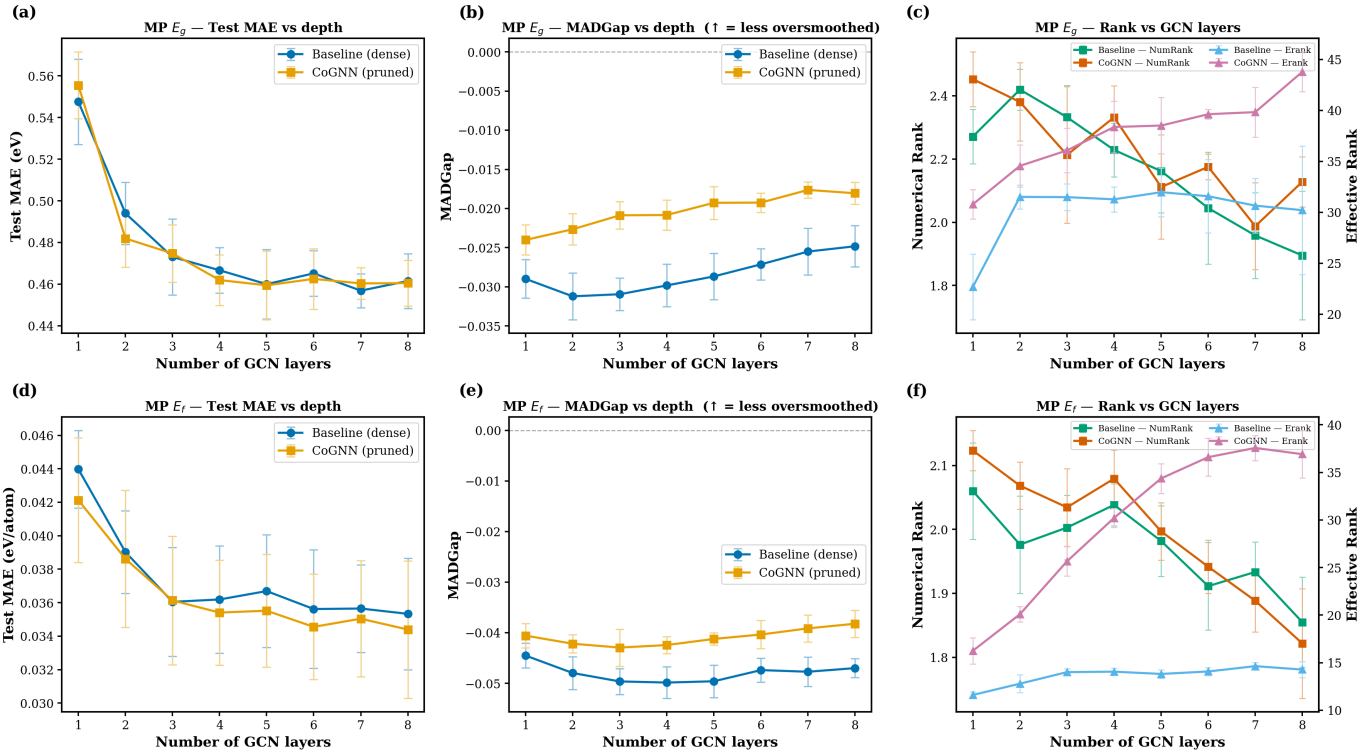


Fig. 3: Combined accuracy and oversmoothing analysis vs. network depth ($L \in \{1, \ldots, 8\}$ GCN layers) for the dense baseline and CoGNN (LEARNABLECUTOFFSCORINGNET action network) on the Materials Project band gap (E_g, top row) and formation energy (E_f, bottom row) tasks. **(a,d)** Test MAE. **(b,e)** MADGap, defined as $\mathrm{MAD}_{\mathrm{remote}} - \mathrm{MAD}_{\mathrm{local}}$ where $\mathrm{MAD} = \mathbb{E}[1 - \cos(h_i, h_j)]$; higher (less negative) values indicate that locally and remotely connected node representations remain more distinguishable, i.e., less over-smoothing. **(c,f)** Numerical rank (left axis) and effective rank (right axis) of the final-layer node embeddings; higher values indicate greater representational diversity and therefore less over-smoothing. Error bars denote the standard deviation across the five cross-validation folds.

MP, which is substantially lower than the 93% and 97% for JARVIS E_g and MP E_g respectively, yet the difference is still withing the standard deviation calculated across the five-folds of the CV scheme, indicating that the structural information preserved in the initial training is sufficient despite not being actively optimizing for E_g.

Performing the experiment in the reverse sequence (using the graph optimized for MP E_g to predict MP E_f) resulted in an MAE $= 0.038 \pm 0.004$ eV/atom, which equivalent to the baseline MAE = 0.036 eV/atom $\pm$ 0.004 eV/atom and to the results for the graph trained directly on E_f (MAE = 0.037 $\pm$ 0.003 eV/atom).

Similar generalization performance has been reported for the JARVIS E_g, E_f as well. While the graphs were trained to predict JARVIS E_f(Phase 1) followed by the training to predict E_g keeping those Phase 1 graphs fixed, the latter model achieved a test MAE of 0.639 ± 0.053 eV, which is very close to the baseline MAE 0.633 eV. The trained graphs for JARVIS E_g shows the MAE of 0.617 ± 0.053 as reported in Table I.

Swapping the source and target task for the JARVIS, the MAE of 0.066 ± 0.006 ev/atom has been reported for the E_f while it was the target task (Phase2). The baseline JARVIS E_f model shows the MAE=0.065 eV/atom. Here, the optimized graphs from the source task(E_g) didn't hurt the predictive performance of phase2 as well. The optimized graph from JARVIS E_f can achieve 0.0621 ± 0.007 eV/atom.

The predictive errors associated with the pruned graphs across different tasks indicate that we can achieve performance comparable to the dense baseline, even when edges are pruned using LEARNABLECUTOFFSCORINGNET optimized for an entirely different task. Furthermore, this demonstrates the network's capacity to identify essential bond interactions successfully cutting down the chemically redundant connections that would otherwise lead the models to sub-optimal performance. As a result, these sparsified graphs can serve as highly effective, generalized structural priors for diverse downstream predictive tasks.

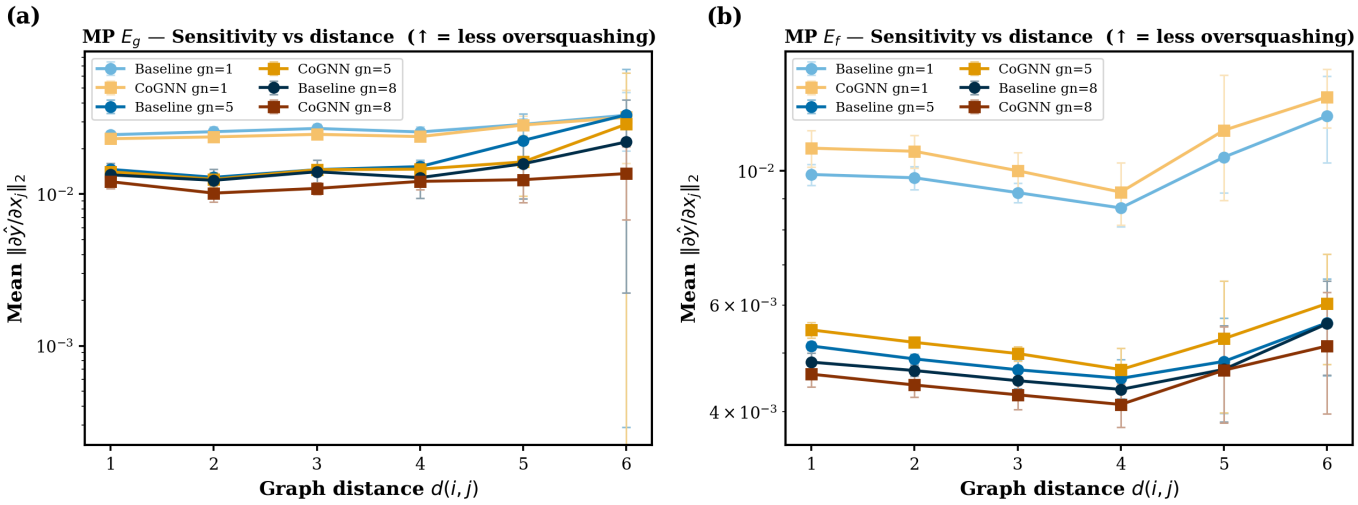


Fig. 4: Over-squashing Analysis via Jacobian Sensitivity on the MP dataset. LEARNABLECUTOFFSCORINGNET has been used as the action network for the CoGNN. Each panel plots the mean Jacobian sensitivity $\bar{S}(d) = \mathbb{E}[\|\partial\hat{y}/\partial x_j\|_2]$ as a function of shortest-path graph distance $d(i,j)$, where $i = \arg\max_j \|\partial\hat{y}/\partial x_j\|_2$ is the most influential atom in each structure. Higher values indicate greater sensitivity to atoms at that distance, i.e. less over-squashing. Results are shown for MP band gap E_g **(left)** and MP formation energy E_f **(right)** at three network depths ($L \in \{1, 5, 8\}$ GCN layers).

TABLE V: Cross-property generalizability of pruned graphs. Target and Source denote the properties used for predicting and pruning, respectively. Transferred MAE and Baseline MAE are reported in eV for E_g and eV/atom for E_f.

Dataset	Target	Source	MAE	Baseline
MP	E_g	E_f	0.511 ± 0.013	0.493 ± 0.017
MP	E_f	E_g	0.038 ± 0.004	0.036 ± 0.004
JARVIS	E_g	E_f	0.639 ± 0.053	0.633 ± 0.047
JARVIS	E_f	E_g	0.066 ± 0.006	0.065 ± 0.006

III. BACKGROUND

A. Crystal graph

A crystal graph $G = (V, E)$ is a multigraph where nodes $v \in V$ represent atoms in the unit cell and directed edges $e = (u, v) \in E$ represent interatomic pairs within a cutoff radius r_{gc}. We use k-nearest-neighbor (k-NN) graph construction with the cut-off radius of 8 Å and maximum 12 number of neighbors, consistent with the ALIGNN baseline [9]. Later, we have tested our approach with different cut off radius and number of neighbors. Each node carries an atomic feature vector obtained from an embedding of the atomic number. Each edge carries a bond feature vector derived from the interatomic distance r_{uv}, expanded with a radial basis function (RBF). Bond lengths r_{uv} are used by the LEARNABLECUTOFFSCORINGNET without any further expansion.

B. CO-GNN

Our cooperative extension of graph neural network has been inspired from the Cooperative Graph Neural Networks (CO-GNNs) [63]. CO-GNN defines four actions-Standard (S), Listen (L), Broadcast (B), and Isolate (I) to formulate the message passing across graph neural network.

Standard (S): Broadcast to neighbors that listen and listen to neighbors that broadcast.

Listen (L): Listen to (receive information from) neighbors that broadcast.

Broadcast (B): Broadcast (send information) to neighbors that listen.

Isolate (I): Neither listen nor broadcast, effectively isolating the node.

In order to assign the actions to each node automatically, CO-GNN is designed to perform two steps of learning; (1) π is a neural network employed to predict the best actions in each layer,

$$\mathbf{p}_v^{(l)} = \pi\big(\mathbf{h}_v^{(l)}, \{\!\{\mathbf{n}_u^{(l)} \mid u \in \mathcal{N}_v\}\!\}\big) \tag{1}$$

where $\mathbf{p}_v^{(l)}$ (`num_nodes` $\times$ 2) is a probability vector used to decide an action for each node, and η in (2) is a neural network employed to update node features upon the actions of each node,

$$\mathbf{h}_v^{(l)} = \begin{cases} \eta^{(l)}\big(\mathbf{h}_v^{(l)}, \{\!\{\}\!\}\big), & a_v^{(l)} = \mathrm{I} \vee \mathrm{B} \\ \eta^{(l)}\big(\mathbf{h}_v^{(l)}, \{\!\{\mathbf{h}_u^{(l)} | u \in \mathcal{N}_v, a_u^{(l)} = \mathrm{S} \vee \mathrm{B}\}\!\}\big), & a_v^{(l)} = \mathrm{L} \vee \mathrm{S}. \end{cases} \tag{2}$$

where $a_v \sim \mathbf{p}_v^{(l)}$ using the Gumbel softmax estimator in 3. The first statement is that if the nodes are assigned to be **I** or **B**, they are updated their-self without aggregating neighbor-node features, and the second statement is that if the nodes are assigned to be **L** or **S**, they are aggregating neighbor-node features and then updated.

C. *Gumbel softmax estimator*

The Gumbel-Softmax estimator (also known as the Concrete distribution) is a technique used in machine learning to allow differentiable sampling from a discrete distribution. It is particularly useful for problems that involve learning categorical variables in models that need to be trained using gradient-based optimization methods, such as neural networks.

$$\text{Gumbel-softmax}(\mathbf{p};\tau) = \frac{\exp\left((\log(\mathbf{p}) + \mathbf{g})/\tau\right.}{\sum_{a\in\Omega}\exp\left((\log(\mathbf{p}(a)) + \mathbf{g}(a))/\tau\right)} \tag{3}$$

where a is an action in a set of actions Ω, $\mathrm{g}(a) \sim \mathrm{GUMBEL}(0,1)$, which is sampled from the Gumbel distribution, and τ is the softmax temperature resulting vectors to a one-hot vector when decreasing. The advantage of Gumbel-softmax estimator is it allows the backpropagation flowing from η network to π network.

IV. Proposed Methods

Our CoGNN is built upon the Edge-Gated Graph Convolution layers (EGNN) [64]. In the standard, EGNN regressor loop, each EGNN layer updates node embeddings H and edge embeddings E_{feat} jointly. Our CoGNN inserts an edge scoring action network preceding the EGNN layers, the action networks work as a function of either isolated edge features, combined edge and node features, or only raw bond length and apply a Gumbel-Softmax activation to output a continuous per-edge soft weight $w_e \in [0,1]$ (see fig 1). This weight is multiplied into the edge gate inside the EGNN to provide numerical advantage to the important bond lengths and disadvantage to the unimportant bond lengths. The architecture can be discussed as below:

1) **Embedding:** Embed atomic numbers and bond distances into initial features $H^{(0)}, E^{(0)}_{\text{feat}}$.
2) **For each GCN layer** $t = 1, \ldots, L$**:**
   a) **Action:** Compute per-edge soft weights $w^{(t)} = \mathrm{ActionNet}(G, H^{(t-1)}, E^{(t-1)}_{\text{feat}}, \mathbf{r})$.
   b) **Environment:** Update features via EGNN with weights $w^{(t)}$: $(H^{(t)}, E^{(t)}_{\text{feat}}) = \mathrm{EGNN}(G, H^{(t-1)}, E^{(t-1)}_{\text{feat}}, w^{(t)})$.
3) **Readout:** Pool node embeddings $H^{(L)}$ to a graph vector h_G and predict $\hat{y} = \mathrm{MLP}(h_G)$.

In atomistic graph neural networks, message passing is bidirectional; each node aggregates information from, and simultaneously broadcasts information to, all of its spatial neighbors. This is because crystal graphs are constructed by connecting all atoms within a specified radial cutoff, yielding an undirected topology. The bidirectional information flow directly corresponds to the **Standard (S)** action discussed in the III-B. Considering this scenario, we modify this conventional message passing by introducing an additional learnable weight that dynamically suppresses redundant or unimportant bonds. This suppression mechanism can isolate a node from the topology by assigning a weight of exactly zero to its connection(edge) and effectively pruning the edge.

In this work, we implement and evaluate four variants of action networks, specifically focusing on their ability to capture the impact of physical bond lengths on property prediction.

A. *Action Networks*

Depending on the architecture, the action networks operate on a selected set of input variables (e.g.node features H, edge features E_{feat}, and raw bond lengths $\mathbf{r}$), and output a per-edge logit that is passed through Gumbel-Softmax to obtain the soft weight w_e. The per edge logit is the two-class unnormalized score from each of the action nets. Out of two classes, the model has been trained to consider the first class as the **keep** and second class as the **drop**. Gumble softmax estimator converts these unnormalized scores into soft-approximation of one hot keep/drop class. After Gumbel-softmax estimation, the first class (keep) is passed as the edge weight to the environment network. We evaluate four designs for action networks.

1) EdgeScoringNet: The EdgeScoringNet concatenates the source-node embedding, destination-node embedding, and edge embedding inspired from Graph Attention Networks [65], then passes them through a learnable scoring head of depth L_a (the number of action network layers). Since all experiments in this work use a single action network layer ($L_a = 1$), this head reduces to a single affine projection:

$$z_e \;=\; W\big[h_{u_e} \,\|\, h_{v_e} \,\|\, e_{u_e v_e}\big] + b \;\in\; \mathbb{R}^2,$$

$$w_e \;=\; \mathrm{GumbelSoftmax}_\tau(z_e)_0,$$

where u_e, v_e are the source and destination of edge e, $W \in \mathbb{R}^{2\times(2d_h+d_e)}$ and $b \in \mathbb{R}^2$ are learnable parameters, d_h and d_e are the node and edge embedding dimensions, and z_e is the two variable logits derived from the EdgeScoringNet. w_e is the derived weight from the action used later in the environment network. For $L_a > 1$ the head becomes an MLP with $L_a - 1$ hidden layers (each a linear layer followed by batch normalization and a SiLU nonlinearity) preceding the final projection to $\mathbb{R}^2$.

2) EdgeOnlyScoringNet: The EdgeOnlyScoringNet is the only edge embedding version of the EdgeScoringNet and ignores node context. The distance only RBF edge features have been used as the input following the SchNet [7]. As in the EdgeScoringNet, the scoring head has depth $L_a = 1$ and therefore reduces to a single affine projection:

$$z_e \;=\; W' e_{u_e v_e} + b' \;\in\; \mathbb{R}^2, \qquad w_e = \mathrm{GumbelSoftmax}_\tau(z_e)_0,$$

where $W' \in \mathbb{R}^{2\times d_e}$ and $b' \in \mathbb{R}^2$ are learnable parameters. This tests whether node context is necessary or whether edge features suffice. The output dimension, the deeper-head generalization for $L_a > 1$, and the later approximation are followed as the EdgeScoringNet.

3) LearnableCutoffScoringNet: This design choice is inspired by the continuous-filter convolutions of SchNet [7], where atomic interactions are modulated by smooth radial basis functions of interatomic distance. This variant replaces the learned filter from SchNet [7] with a *learned distance threshold*. The key insight from SchNet is that interatomic distance is the dominant physical predictor of bond relevance. In this network, we have used this raw distance as input to capture the bond relevance without any atom-type context.

The scoring function is parameterized as:

$$\text{logit}_e \;=\; \frac{|\, r_{\text{cut}} \,| - r_e}{\text{clamp}(|\tau|,\, \text{min} = 0.05)}, \tag{4}$$

where r_e is the bond length, r_{cut} is a scalar learnable parameter initialized to r_0 (default 3.5 Å), and τ is a scalar softness parameter initialized to 0.5. Clamping $|\tau|$ prevents numerical collapse when $\tau \to 0$.

The network returns logits as the symmetric pair $[\text{logit}_e, -\text{logit}_e]$, which are passed through the Gumbel-Softmax path with temperature τ_{ext} (distinct from the learnable τ in (4)). These symmetric logits impose a strong bias towards the expected bond lengths, which needs to be kept. As the first class contains the **keep** logits, the positive values carry a strong signal of the corresponding bond distances prior to the Gumble-softmax estimation. Since the two columns are constrained as $[a, -a]$, Gumbel-Softmax on column 0 reduces to $\sigma(2a/\tau_{\text{ext}})$—a monotone function of a alone—so the stochastic noise does not alter the ranking of edges derived from the edge scoring net and has no significant effects on which bonds are pruned.

The learned r_{cut} converges to the data-driven optimal cutoff radius, and the entire variant has only 2 scalar parameters, making it both highly regularized and highly interpretable.

4) MultiplicativeGateScoringNet: This variant combines a node-level score with an edge-level gate multiplicatively:

First, a geometry gate is computed from the bond embedding e_{ij} alone via a small MLP followed by a sigmoid activation:

$$g_e = \sigma(W_2\, \text{SiLU}(W_1\, e_{ij} + b_1) + b_2) \in [0, 1] \tag{5}$$

Secondly, a node score is computed from the concatenated source and destination node embeddings.

$$s_e = W_4\, \text{SiLU}(\text{BN}(W_3\, [h_{u_e} \,\|\, h_{v_e}] + b_3)) + b_4 \tag{6}$$

where u_e and v_e are the source and destination atoms of edge e, $[\cdot \,\|\, \cdot]$ denotes concatenation, and BN denotes batch normalization.

The final per-edge logit is formed as the element-wise product of the geometry gate and the node score:

$$\text{logit}_e = g_e \cdot s_e \tag{7}$$

MultiplicativeGateScoringNet prioritizes both the atomic information and the bond information. The multiplication between the geometry gate and the node score couldn't allow to achieve a higher logit when either the gate score or the node score is low. In this way, the bond retention has been aimed to get determined by the geometric and chemical information.

The per-edge logit is then wrapped in the same symmetric two-logit construction used by LEARNABLECUTOFFSCORINGNET:

$$\mathbf{z}_e = [\,\text{logit}_e,\; -\text{logit}_e\,] \in \mathbb{R}^2 \tag{8}$$

and passed through Gumbel-Softmax to obtain the soft edge weight $w_e = \text{GumbelSoftmax}(\mathbf{z}_e;\, \tau)_0$

B. Environment Network

Environment network is the traditional regressor network. It has been designed to perform one additional weight calculation from the predicted actions of action network. This weight is used in the message passing alongside initial node features and edge features. As node feature, atomic number has been used while interatomic distance has been used as edge features. The functionality of action network can be categorized into four parts:

- Form per-edge weights by node-wise product
- Repeat Edge-Gated Convolution for the final Graph Level convolution
- Readout (graph-level embedding)
- Graph Level Prediction via MLP

*a) **Form per-edge weights by node-wise product**:* Let edge index vector be $(u, v) = \text{edges}(G)$, where u_e is source and v_e is destination for edge e.

$$w_e \;=\; p^{\text{in}}_{v_e} \cdot p^{\text{out}}_{u_e}, \qquad \forall e \in E.$$

Each edge weight is the product of the destination-node's in-keep probability(appearance) and the source-node's out-keep probability(appearance).

*b) **Repeat Edge-Gated Convolution for the final Graph Level convolution**:* If the model has L stacked EdgeGatedConv layers, we can denote:

$$(H^{(t+1)}, E^{(t+1)}) \;=\; \text{EdgeGatedConv}_{\theta^{(t)}}\big(G, H^{(t)}, E^{(t)}, w\big)$$

$$\text{for } t = 0, \ldots, L-1.$$

Here each layer uses the w which has been computed from the previous step.

*c) **Readout (graph-level embedding)**:* After the final GNN layer produce node embeddings $H^{(L)}$ the graph embedding is computed.

$$h_G \;=\; \text{Readout}(G, H^{(L)}) \in \mathbb{R}^{d_{\text{readout}}}$$

*d) **Graph Level Prediction via MLP**:*

$$\hat{y} \;=\; \text{MLP}_\Phi(h_G) \;=\; (\text{FC}^{(K)} \circ \cdots \circ \text{FC}^{(1)})(h_G)$$

*1) **Edge Gated Graph Convolution Net**:* Edge gated graph convolution updates node representations h^l from layer l according to the following formula:

For each edge $e : (j \to i)$ compute the edge pre-activation:

$$m_{ij} \;=\; W_s h_j + W_d h_i + W_e e_{ij}$$

compute elementwise gate:

$$\sigma_{ij} \;=\; w_{ij} \cdot \mathrm{sigmoid}\left(m_{ij}\right)$$

project source node values for messages:

$$\widetilde{h}_j \;=\; W_v h_j$$

edge-modulated message along $(j \to i)$:

$$\mu_{ij} \;=\; \sigma_{ij} \odot \widetilde{h}_j$$

aggregate per-destination:

$$A_i \;=\; \sum_{j \in \mathcal{N}(i)} \mu_{ij}, \qquad S_i \;=\; \sum_{j \in \mathcal{N}(i)} \sigma_{ij}$$

normalized neighbor contribution:

$$h'_i \;=\; \frac{A_i}{S_i + \varepsilon}$$

combine with self-term and nonlinearity:

$$x_i \;=\; W_u h_i + h'_i, \qquad \tilde{x}_i \;=\; \mathrm{SiLU}\big(\mathrm{BN}_{\text{nodes}}(x_i)\big)$$

optionally residualize:

$$H_i^{(1)} \;=\; h_i + \tilde{x}_i \quad \text{(if residual)}$$

edge update:

$$y_{ij} \;=\; \mathrm{SiLU}\big(\mathrm{BN}_{\text{edges}}(m_{ij})\big), \qquad E_{ij}^{(1)} \;=\; e_{ij} + y_{ij} \quad \text{(if residual)}$$

C. Loss Function with Sparsity Regularization

The training loss combines a property prediction term with a sparsity penalty:

$$\mathcal{L} = \underbrace{\mathrm{MAE}(y, \hat{y})}_{\text{property prediction}} + \lambda \cdot \underbrace{\frac{1}{|E|} \sum_{e \in E} w_e}_{\text{sparsity penalty}}, \tag{9}$$

where $\lambda > 0$ is the sparsity weight (hyperparameter denoted `sw`). The sparsity penalty penalizes the soft edge weights by assigning lower values. The objective is to enable the model to justify its decision about the edges, it decided to keep. Specifically, if the action network decides to keep all the edges, then the regularization term will contribute more to the loss, therefore in the backpropagation the signal towards the action net will be the higher number of edges is detrimental. After that when the model starts pruning edges, the final loss might increase because of losing information. So, it's a kind of learning from the tradeoff situation where the model is trying to reach an optimal state where the missing edges won't impact the final MAE.

V. Experimental Setup

A. Dataset

To evaluate our proposed architecture, we train and test our models on two widely adopted material databases: the Materials Project [58] and the JARVIS-DFT database [57]. Standard benchmarking in the field of graph based material informatics includes small unit cell which doesn't create significant overhead in the computation and representation during message passing. Therefore, we have emphasized over the subset of large unit cell to showcase the efficacy of our dynamic pruning mechanism and its ability to mitigate topological noise.

Standard graph construction protocol following fixed size cut off radius(eg 8.0Å) and maximum number of neighbors(eg 12) can create problems for large unit cell by generating geometrically proximal but chemically irrelevant contacts. This is the outcome of increasing number of edges while the number of atoms grow. This effects can be compounded with the increasing number of graph convolution layers and hops making the large crystal structures as the ideal testbed for the CoGNN.

To demonstrate whether the CoGNN is enable to filter out the physically meaningful topology out of dense, noisy graph we extracted a subset of 2754 larger structures from Materials Project containing ≥ 140 atoms per unit cell, and 989 complex structures from the JARVIS database containing ≥ 40 atoms per unit cell where edge density is high.

As we need to specify a task to evaluate the performance, the task will be specific property such as bandgap, formation energy of crystal. In this work, all variants of model have been trained to predict formation energy and bandgap. Early study suggests that formation energy is highly affected by the local crystal environment. On the contrary, bandgap is an extended electronic property that depends on the long range periodicity and the extended structural topology of the crystal lattice.

B. Architecture

We train two neural networks jointly within the same pipeline. The first network (the Action Network) is trained to learn learn the bond relevance while the second network(Environment Network) utilizes this topological information to perform the message passing and final prediction. Both networks are optimized simultaneously to minimize a total loss function comprising the Mean Absolute Error (MAE) and a sparsity penalty.

For all primary experiments, we employ one Action Network layer and two Environment Network layers. However, in case of over-smoothing and over-squashing analysis, the number of environment network layers is varied from 1 to 8.

Each action network assigns a continuous survival weight $w_e \in [0, 1]$, to every edge. To convert these continuous scores into explicit pruning decisions, we apply a hard threshold θ. Edges where $w_e < \theta$ are multiplicatively masked to zero (effectively pruned), whereas edges with $w_e \geq \theta$ retain their learned weights. Here,θ serves as a fixed hyperparameter rather

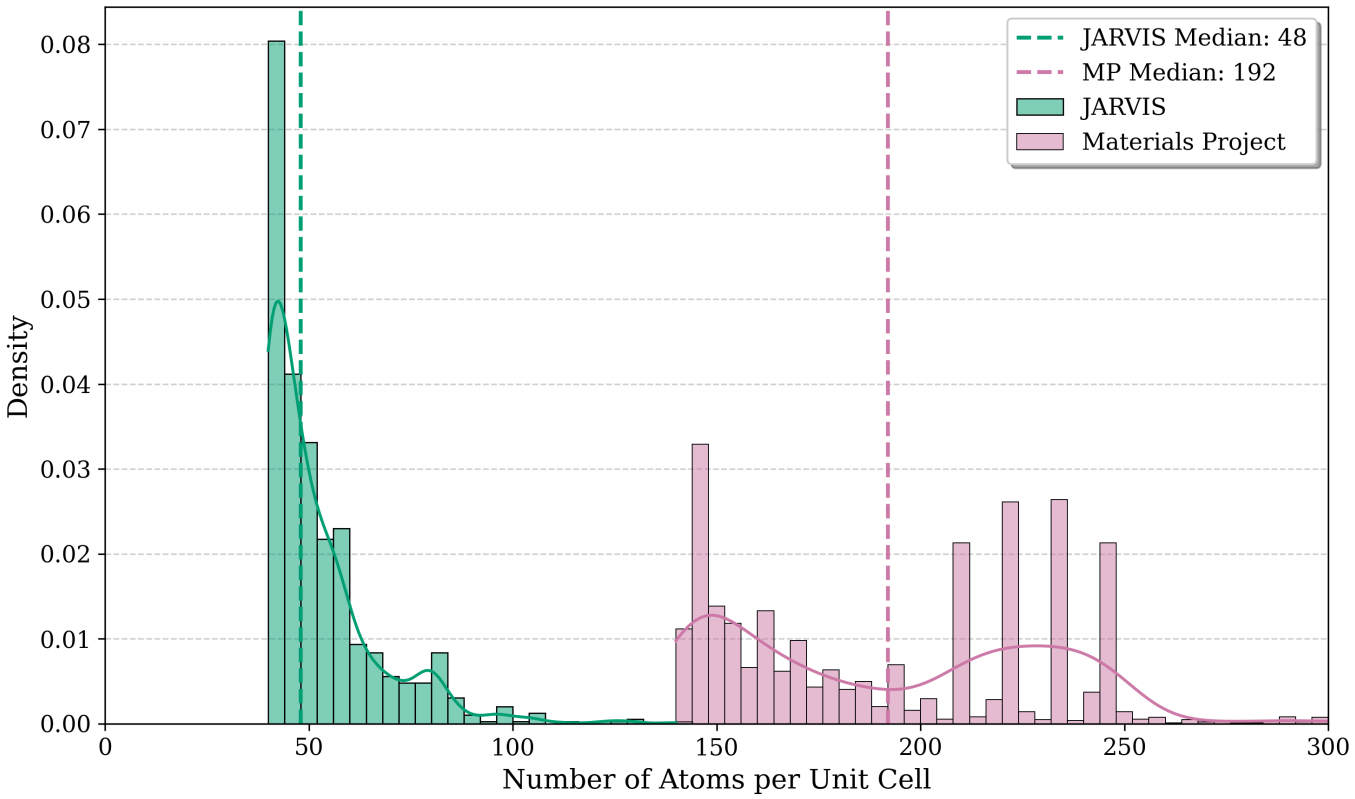


Fig. 5: Distribution of the number of atoms per unit cell for the JARVIS and Materials Project (MP) datasets. The Kernel Density Estimation(KDE) and histograms illustrate the structural variations of the materials evaluated in this study. The JARVIS dataset(green) consists of smaller unit cells with a median of 48 atoms. In contrast, the MP dataset(pink) demonstrates a much wider distribution with significantly larger structures having a median of 192 atoms. The large MP structures pool create the topological requirements to investigate the over-smoothing and over-squashing across larger dense graphs.

than a learned parameter, it is not updated by gradient descent. Instead, we sweep over $\theta \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$ across separate training runs, and the best-performing θ is reported for each configuration. We apply θ identically during both training and evaluation to investigate what degree of pruning can be achieved without compromising the predictive accuracy.

C. Cross-Validation Protocol

All results reported in this work use 5-fold cross-validation protocol, applied consistently across every training scheme, dataset, and target property. For each dataset-target combination, the crystal structures are shuffled using a fixed random seed and partitioned into five mutually exclusive folds via the `scikit-learn KFold` utility. During each of the five iterations, one fold(20% of the dataset) is held out for testing, while the remaining four folds(80%) are sub-divided into a 95%/5% training and validation split. This systematic rotation ensures that every structure is evaluated as a test sample exactly once. Final performance metrics—including test MAE, AERR, MADGap, and rank statistics—are calculated independently for each test fold and reported as the mean $\pm$ standard deviation.

D. Training

Three distinct training pipelines were executed to study different aspects of the model. All three share the same optimizer configuration: **AdamW** with learning rate 10^{-3} and weight decay 10^{-5}, combined with a **OneCycleLR** scheduler over 750 epochs with batch size 32. Training loss is $\mathcal{L} = \text{MAE} + \lambda\bar{w}$ (Eq. 9); validation and test performance use MAE only.

1) Primary Action Network Comparison: We have identified the best-performing action network variant across both datasets and both targets. In this training scheme, all four variants of action networks have been used separately, additionally the no-action network baseline has been trained. The crystal graph has been constructed with $r_{\text{gc}} = 8$ Å, $k = 12$ neighbors. For the LEARNABLECUTOFFSCORINGNET: initial cutoff $r_0 = 3.5$ Å, initial softness $\tau_0 = 0.5$ has been set where sparsity weight $\lambda \in \{0.001, 0.01\}$. At the training and evaluation, threshold $\theta \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$ is swept; results are reported at the best θ with $\lambda = 0.001$.

The four variant of action nets aim to find a optimal learning scenario for the model. In this objective, EdgeScoringNet has been designed to use the context from both node and

edge embedding via a learnable MLP. On the contrary, to incorporate physical constraints like bond length, LearnableCutoffScoringNet has been designed using only bond lengths with two scalar parameters. To provide more expressive power, MultiplicativeGateScoringNet has been built upon a distance gate to capture the geometry and a node par score to learn the chemistry from the atom-type context. We have kept the graph construction parameters fixed for all four variants to investigate the impact of each action network design.

All four variants share a unified inference path: each network outputs per-edge logits of shape $[E, 2]$ (keep vs. drop), and edge weights are obtained via Gumbel-Softmax with temperature τ. For EdgeScoringNet and EdgeOnlyScoringNet, both logit columns are independently learned, making the keep/drop decision a genuine 2-class categorical choice that benefits from the stochastic relaxation Gumbel-Softmax provides. For LearnableCutoffScoringNet and MultiplicativeGateScoringNet, the two columns are constrained as $[a, -a]$—a symmetric pair encoding a single scalar decision—so Gumbel-Softmax on column 0 reduces to a monotone function of the underlying scalar logit, and the stochastic relaxation has no effect on the learned ranking of edges. The unified code path ensures all variants are evaluated under identical conditions.

2) Graph Construction Parameter Sweep: In this scheme, we have determined whether the density and extent of the input crystal graph affects LearnableCutoffScoringNet performance, and understand how the action network interacts with different neighbor pools.

We have only used LEARNABLECUTOFFSCORING Net in this training scheme as its the best performing action net from the first pipeline.

- Graph construction configurations setup:

r_{gc} (Å)	k (max neighbors)
5	8
5	12
6	8
6	12
8	8
8	12
8	16
8	20
10	24

- Initial cutoff fixed at $r_0 = 3.5$ Å for all configurations (to isolate graph-construction effects from learned-cutoff initialization).
- Threshold $\theta \in \{0.2, 0.3, 0.4, 0.5\}$; $\lambda = 0.001$.
- Datasets: JARVIS (E_g, E_f) and MP (E_g, E_f).

The graph construction allows the the LearnableCutoffScoringNet to only assign scores to bonds that exist in the input k-NN graph;it cannot recover a bond that was excluded during preprocessing. Considering this constraint, the graph is constructed with a large cutoff (e.g., $r_{\mathrm{gc}} = 10$ Å, $k = 24$), it includes many weak long-range connection; in that case action network will get a large candidate pool of edges to prune, without the action network, the subsequent model needs to navigate a denser, noisier message passing graph during training.

Conversely, a tight construction ($r_{\mathrm{gc}} = 5$ Å, $k = 8$) would remove many important bonds during prepossessing. These sparser graphs can affect the both bandgap and formation energy. We have kept the $r_0 = 3.5$ Å for the LearnableCutoffScoringNet across all graph configurations. The objective was to capture the only influence of different neighbor pool and graph density in the MAE.

This design choice will expand the study of whether the optimal pruning can be effective regardless of the size of the topology.

3) Initial r_{cut} Sensitivity Sweep: This training scheme aims to test the convergence of the learned distance threshold whether the initialization matters verifying the predictive performance and edge-pruning distribution across different r_{cut}. The following configurations have been followed out to perform this training.

- Action net: LearnableCutoffScoringNet only.
- Graph construction: ($r_{\mathrm{gc}} = 8$ Å, $k = 12$).
- Initial cutoff $r_0 \in \{2.0, 2.5, 3.0, 3.5, 4.0, 4.5\}$ Å; $\tau_0 = 0.5$ in all cases.
- Threshold $\theta \in \{0.1, 0.2, 0.3, 0.4, 0.5\}$; $\lambda \in \{0.001\}$.
- Datasets: JARVIS (E_g, E_f) and MP (E_g, E_f).

The LearnableCutoffScoringNet parameterizes the pruning decision as

$$\mathrm{logit}_e = \frac{|r_{\mathrm{cut}}| - r_e}{\mathrm{clamp}(|\tau|,\, 0.05)},$$

where r_{cut} and τ are jointly trained by gradient descent. If the training loss is convex in r_{cut}, that implies a single chemically optimal cutoff exists for the dataset. Under this condition, models initialized at different r_0 should converge to the same r^*_{cut} regardless of starting point. This set of experiments aim to investigate how the initial cutoff triggers different failure modes within the LearnableCutoffScoringNet. First, initializing $r_0 = 2.0$ Å (well inside the covalent bond range) means that virtually all bonds are initially assigned large positive logits (nearly kept), so the action network produces near-uniform soft weights from the very first epoch. In this scenario, the gradient from the sparsity penalty $\lambda\bar{w}$ drives r_{cut} toward even shorter values (to reduce mean weight), potentially trapping the model in a regime where pruning is minimal because the logit is always large. Second, initializing $r_0 = 4.0$ Å (longer than most covalent bonds but still within the 8 Å graph cutoff) starts the model by keeping a larger fraction of bonds; the MAE loss then provides the gradient signal to reduce r_{cut} toward the true covalent cutoff. Third, if the final r^*_{cut} depend on the initialization, that means the model finds different local minima for different initial cut off. Consequently, the pruning decision of the network will be different as well that is highly unlikely. This sweep therefore serves a dual purpose: (1) validating the robustness of the learned cutoff across different set of initialization, and (2) providing practical guidance—if results are invariant to $r_0 \in [2.0, 4.5]$ Å, users can safely use the default $r_0 = 3.5$ Å.

4) Network Depth Sweep for Over-smoothing and Over-squashing: Both over-smoothing and over-squashing investigations depend on the network depth; therefore, in this training scheme the number of message-passing layers has been varied while holding all other factors fixed. In this pipeline, we therefore train an independent model at each environment network depth $L \in \{1, 2, \ldots, 8\}$, for the dense baseline and for CoGNN with the LEARNABLECUTOFFSCORINGNET, on the Materials Project E_g and E_f tasks. Only one action network layer has been used throughout all experiments. The crystal graph has been constructed with $r_{\text{gc}} = 8$ Å, $k = 12$ neighbors while $r_0 = 3.5$ Å, $\tau_0 = 0.5$ have been used for LEARNABLECUTOFFSCORINGNET and $\lambda = 0.001$ has been used for the sparsity regularizer in the loss function. The best-performing θ from pipeline 1 has been used for pruning for each training and evaluation. This pipeline yields a total $8 \times 2 \times 2 = 32$ independently trained configurations, each evaluated over five folds. The metrics used to evaluate over-smoothing and over-squashing require no additional training: MADGap, the rank-based metrics, and the Jacobian sensitivity are all computed by inference on the already-trained checkpoints. For each fold, we register a forward hook on the final graph convolution layer to extract the node embedding matrix $X^{(L)}$ of every test structure, from which MAD, MADGap, $\mathrm{NumRank}$, and Erank are computed as defined in Sections V-E4 and V-E5. MAD and the rank metrics use all test structures in the fold, aggregated as a node-count-weighted mean. The Jacobian sensitivity of Section V-E3 is more expensive, as it requires a backward pass per structure; it is therefore computed on 60 randomly selected test structures per fold, with shortest-path distances truncated at $d \leq 6$. All reported curves are the mean $\pm$ standard deviation of these per-fold quantities across the five folds.

5) Cross-Property Generalizability: This final pipeline aims to test whether a topology pruned for one property remains useful for a different property; specifically, we have investigated whether the action network has learned transferable chemistry rather than target-specific shortcuts. The experiment proceeds in two phases.

In **Phase 1**, a full CoGNN model with the LEARNABLECUTOFFSCORINGNET is trained on a *source* property using $r_0 = 3.5$ Å, $\tau_0 = 0.5$. The crystal graph has been constructed with $r_{\text{gc}} = 8$ Å, $k = 12$ neighbors while $\lambda = 0.001$ has been used for the sparsity regularizer in the loss function. The learned per-edge weights are then thresholded at the optimal θ identified from the V-D1 for that source task, and the resulting sparsified graphs are exported and frozen. In **Phase 2**, a plain baseline model *without any action network* is trained from scratch on a different *target* property. In this phase, only the frozen graphs from phase 1 are fed to the model, therefore no explicit graph construction is required and the sparsity penalty is disabled ($\lambda = 0$); the loss reduces to MAE alone.

Both transfer directions are evaluated on both datasets: $E_f \to E_g$ and $E_g \to E_f$ for MP and JARVIS. Each transferred model is compared against two references: the dense-graph baseline for the target property, and the CoGNN model whose topology was optimized directly for that target property.

E. Evaluation Metric

1) Mean Absolute Error: The primary evaluation metric is **test MAE** (mean absolute error, eV for E_g and eV/atom for E_f), computed as the mean and standard deviation across the five outer folds of cross-validation. The threshold θ is swept at both training and evaluation time (not backpropagated through) and the best-performing θ is reported for each variant.

2) Average Edge Retention Rate: In addition to MAE, we track the **Average Edge Retention Rate(AERR)**: the fraction of training and test-set edges whose soft weight $w_e \geq \theta$ after pruning, which quantifies how aggressively the action network prunes the graph at a given threshold. This secondary metric is essential because two models with identical MAE but very different edge retention rates carry different interpretations—a model that retains 30% of edges is more informative about which bonds are chemically irrelevant than one that retains 98% of edges.

3) Jacobian Sensitivity: To measure the over-squashing, the sensitivity metric, **Jacobian Sensitivity**, is computed as follows. Formally, for an atom j with input embedding x_j, we compute

$$S_j = \left\| \frac{\partial \hat{y}}{\partial x_j} \right\|_2, \tag{10}$$

For each test structure, we identify a reference atom $i = \arg\max_j S_j$ i.e. the node with maximal influence on the prediction and compute the shortest-path graph distance $d(i, j)$ from this reference to every other atom j in the structure. Sensitivities are aggregated by the $d(i, j)$ and later averaged across all test structures to obtain the mean sensitivity curve $\bar{S}(d)$.

4) Mean Average Distance: Mean Average Distance(MAD) has been used to measure the over-smoothing.
Let $h_i \in \mathbb{R}^H$ denote the final-GCN-layer embedding of atom i in a crystal graph with N atoms. For any two atoms $i \neq j$, define their pairwise dissimilarity as one minus the cosine similarity of their embeddings,

$$D_{ij} = 1 - \frac{h_i \cdot h_j}{\|h_i\| \, \|h_j\|}, \qquad D_{ij} \in [0, 2]. \tag{11}$$

D_{ij} is small when two atoms' representations point in nearly the same direction (i.e., they have become hard to distinguish) and large when their representations remain distinct.

The Mean Average Distance over all atom pairs in a structure is the average of D_{ij} over every pair,

$$\mathrm{MAD}_{\text{all}} = \frac{1}{N(N-1)} \sum_{i \neq j} D_{ij}. \tag{12}$$

This average is further split according to whether a pair of atoms is directly bonded in the crystal graph. Let $\mathcal{E}$ denote the set of bonded (edge) atom pairs. The *local* MAD restricts

the average to bonded pairs, and the *remote* MAD restricts it to non-bonded pairs,

$$\mathrm{MAD}_{\text{local}} = \frac{1}{|\mathcal{E}|}\sum_{(i,j)\in\mathcal{E}} D_{ij}, \qquad \mathrm{MAD}_{\text{remote}} = \frac{1}{|\overline{\mathcal{E}}|}\sum_{(i,j)\notin\mathcal{E},\, i\neq j} D_{ij}. \tag{13}$$

Finally, the MADGap is the difference between the two,

$$\mathrm{MADGap} = \mathrm{MAD}_{\text{remote}} - \mathrm{MAD}_{\text{local}}. \tag{14}$$

A larger (more positive, or in our case less negative) MADGap means bonded and non-bonded atoms remain more distinguishable from one another — i.e., less oversmoothing — while $\mathrm{MADGap} \to 0$ signals that locally and remotely connected atoms have become equally (in)distinguishable, the signature of oversmoothing [59].

For each depth L and each model, $\mathrm{MAD}_{\text{all}}$, $\mathrm{MAD}_{\text{local}}$, $\mathrm{MAD}_{\text{remote}}$, and MADGap are computed per test structure from Equations (11)–(14), averaged across test structures (weighted by atom count N) within each of the 5 cross-validation folds, and reported as the mean $\pm$ standard deviation of these per-fold values.

5) Rank: In addition to MAD, we have examined the numerical and effective rank to measure the over-smoothing. For a trained model of depth L, let $X^{(L)} \in \mathbb{R}^{N\times H}$ denote the matrix of node embeddings extracted from the final GCN layer for a crystal graph with N atoms, where each row is the H-dimensional representation of one atom. The (linear-algebraic) rank of $X^{(L)}$ is the number of linearly independent rows (equivalently, columns) of the matrix – the number of genuinely distinct directions of variation present among the node representations. In the extreme case of complete oversmoothing, every row of $X^{(L)}$ becomes a scalar multiple of the same vector, and $X^{(L)}$ collapses to rank 1.

Because matrix rank is a discrete quantity, it changes only when a row becomes an *exact* linear combination of the others, and is therefore insensitive to the gradual, approximate collapse that occurs in practice. We instead use two continuous relaxations of rank, both derived from the singular value decomposition (SVD) of $X^{(L)}$,

$$X^{(L)} = U\Sigma V^{\top}, \tag{15}$$

where $U \in \mathbb{R}^{N\times N}$ and $V \in \mathbb{R}^{H\times H}$ are orthogonal and $\Sigma \in \mathbb{R}^{N\times H}$ is diagonal with non-negative entries $\sigma_1 \geq \sigma_2 \geq \cdots \geq \sigma_r \geq 0$, $r = \min(N, H)$, the *singular values* of $X^{(L)}$, sorted in decreasing order.

a) Frobenius and spectral norms.: The squared Frobenius norm of $X^{(L)}$ is the sum of the squares of all its entries, and is related to the singular values by

$$\|X^{(L)}\|_F^2 = \sum_{i=1}^{N}\sum_{j=1}^{H}\left(X_{ij}^{(L)}\right)^2 = \sum_{k=1}^{r}\sigma_k^2, \tag{16}$$

while the spectral (operator) norm is the largest singular value,

$$\|X^{(L)}\|_2 = \sigma_1. \tag{17}$$

b) Numerical rank.: The numerical rank is the ratio of the total squared "energy" of the representation matrix to the energy captured by its single dominant direction,

$$\mathrm{NumRank}\Big(X^{(L)}\Big) = \frac{\|X^{(L)}\|_F^2}{\|X^{(L)}\|_2^2} = \frac{\sum_{k=1}^{r}\sigma_k^2}{\sigma_1^2}. \tag{18}$$

By construction, $1 \leq \mathrm{NumRank}(X^{(L)}) \leq r$: the lower bound $\mathrm{NumRank} = 1$ is attained exactly when $X^{(L)}$ is rank 1 ($\sigma_2 = \cdots = \sigma_r = 0$), and the upper bound $\mathrm{NumRank} = r$ is attained when all singular values are equal ($\sigma_1 = \cdots = \sigma_r$), i.e. representational energy is spread perfectly evenly across all r directions.

c) Effective (Shannon) rank.: The effective rank instead treats the normalized singular-value spectrum as a probability distribution and measures its entropy. Define

$$p_k = \frac{\sigma_k}{\sum_{j=1}^{r}\sigma_j}, \qquad k = 1, \ldots, r, \tag{19}$$

so that $p_k \geq 0$ and $\sum_{k=1}^{r} p_k = 1$. The Shannon entropy of this distribution is $H(p) = -\sum_{k=1}^{r} p_k \log p_k$ (with the convention $0 \log 0 = 0$), and the effective rank is its exponential,

$$\mathrm{Erank}\Big(X^{(L)}\Big) = \exp\left(-\sum_{k=1}^{r} p_k \log p_k\right). \tag{20}$$

As with $\mathrm{NumRank}$, $1 \leq \mathrm{Erank}(X^{(L)}) \leq r$: $\mathrm{Erank} = 1$ when a single $p_k = 1$ (rank-1 collapse, zero entropy), and $\mathrm{Erank} = r$ when $p_1 = \cdots = p_r = 1/r$ (maximum entropy, energy spread perfectly evenly). Unlike $\mathrm{NumRank}$, which is most sensitive to how dominant the single leading direction is, Erank is sensitive to the shape of the *entire* spectrum, including how energy is distributed among the smaller, secondary directions.

d) Aggregation.: For each test-set structure, $\mathrm{NumRank}$ and Erank are computed from Equations (18) and (20) using the final-layer node embeddings extracted via the same forward-hook procedure used for the MAD computation (Section V-E4). Per-fold values are obtained as the node-count-weighted mean over all test structures in that fold; the values reported in Figure **??** are the mean $\pm$ standard deviation of these per-fold values across the 5 cross-validation folds, for each network depth $L \in \{1, \ldots, 8\}$ and each model (dense baseline vs. CoGNN with LEARNABLECUTOFFSCORINGNET).

VI. CONCLUSION

In this work, we have introduced additional action networks to guide traditional regressor networks in selecting important connections. One of the bottlenecks of crystal structures is the bond quantization. The addition of the action network can help to identify a set of important interatomic bonds which are sufficient for the electronic properties like band gap and formation energy compared to the all bonds of the crystal structure. The efficacy of the action network has been found in removing a certain percentage of edges, yet it leads

to similar performance. The pruned graphs can be used for other tasks different from the task for which they have been optimized. This indicates the model's ability to capture the physical constraints of the crystal structures.

REFERENCES

[1] Peter W Battaglia, Jessica B Hamrick, Victor Bapst, Alvaro Sanchez-Gonzalez, Vinicius Zambaldi, Mateusz Malinowski, Andrea Tacchetti, David Raposo, Adam Santoro, Ryan Faulkner, et al. Relational inductive biases, deep learning, and graph networks. arxiv 2018. *arXiv preprint arXiv:1806.01261*, 10, 2018.

[2] Patrick Reiser, Marlen Neubert, André Eberhard, Luca Torresi, Chen Zhou, Chen Shao, Houssam Metni, Clint van Hoesel, Henrik Schopmans, Timo Sommer, et al. Graph neural networks for materials science and chemistry. *Communications Materials*, 3(1):93, 2022.

[3] David K Duvenaud, Dougal Maclaurin, Jorge Iparraguirre, Rafael Bombarell, Timothy Hirzel, Alán Aspuru-Guzik, and Ryan P Adams. Convolutional networks on graphs for learning molecular fingerprints. *Advances in neural information processing systems*, 28, 2015.

[4] Christopher Sutton, Luca M Ghiringhelli, Takenori Yamamoto, Yury Lysogorskiy, Lars Blumenthal, Thomas Hammerschmidt, Jacek R Golebiowski, Xiangyue Liu, Angelo Ziletti, and Matthias Scheffler. Crowd-sourcing materials-science challenges with the nomad 2018 kaggle competition. *npj Computational Materials*, 5(1):111, 2019.

[5] Tian Xie and Jeffrey C Grossman. Crystal graph convolutional neural networks for an accurate and interpretable prediction of material properties. *Physical review letters*, 120(14):145301, 2018.

[6] Chi Chen, Weike Ye, Yunxing Zuo, Chen Zheng, and Shyue Ping Ong. Graph networks as a universal machine learning framework for molecules and crystals. *Chemistry of Materials*, 31(9):3564–3572, 2019.

[7] Kristof T Schütt, Huziel E Sauceda, P-J Kindermans, Alexandre Tkatchenko, and K-R Müller. Schnet–a deep learning architecture for molecules and materials. *The Journal of Chemical Physics*, 148(24), 2018.

[8] Kristof Schütt, Oliver Unke, and Michael Gastegger. Equivariant message passing for the prediction of tensorial properties and molecular spectra. In *International Conference on Machine Learning*, pages 9377–9388. PMLR, 2021.

[9] Kamal Choudhary and Brian DeCost. Atomistic line graph neural network for improved materials property predictions. *npj Computational Materials*, 7(1):185, 2021.

[10] Johannes Gasteiger, Florian Becker, and Stephan Günnemann. Gemnet: Universal directional graph neural networks for molecules. *Advances in Neural Information Processing Systems*, 34:6790–6802, 2021.

[11] Johannes Gasteiger, Janek Groß, and Stephan Günnemann. Directional message passing for molecular graphs. *arXiv preprint arXiv:2003.03123*, 2020.

[12] Johannes Gasteiger, Shankari Giri, Johannes T Margraf, and Stephan Günnemann. Fast and uncertainty-aware directional message passing for non-equilibrium molecules. *arXiv preprint arXiv:2011.14115*, 2020.

[13] Shufeng Kong, Francesco Ricci, Dan Guevarra, Jeffrey B Neaton, Carla P Gomes, and John M Gregoire. Density of states prediction for materials discovery via contrastive learning from probabilistic embeddings. *Nature communications*, 13(1):949, 2022.

[14] Mario Geiger and Tess Smidt. e3nn: Euclidean neural networks. *arXiv preprint arXiv:2207.09453*, 2022.

[15] Zhantao Chen, Nina Andrejevic, Tess Smidt, Zhiwei Ding, Qian Xu, Yen-Ting Chi, Quynh T Nguyen, Ahmet Alatas, Jing Kong, and Mingda Li. Direct prediction of phonon density of states with euclidean neural networks. *Advanced Science*, 8(12):2004214, 2021.

[16] Steph-Yves Louis, Yong Zhao, Alireza Nasiri, Xiran Wang, Yuqi Song, Fei Liu, and Jianjun Hu. Graph convolutional neural networks with global attention for improved materials property prediction. *Physical Chemistry Chemical Physics*, 22(32):18141–18148, 2020.

[17] Prathik R Kaundinya, Kamal Choudhary, and Surya R Kalidindi. Prediction of the electron density of states for crystalline compounds with atomistic line graph neural networks (alignn). *JOM*, 74(4):1395–1405, 2022.

[18] Ilyes Batatia, David P Kovacs, Gregor Simm, Christoph Ortner, and Gabor Csanyi. Mace: Higher order equivariant message passing neural networks for fast and accurate force fields. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 11423–11436. Curran Associates, Inc., 2022.

[19] Larry Zitnick, Abhishek Das, Adeesh Kolluru, Janice Lan, Muhammed Shuaibi, Anuroop Sriram, Zachary Ulissi, and Brandon Wood. Spherical channels for modeling atomic interactions. In S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, editors, *Advances in Neural Information Processing Systems*, volume 35, pages 8054–8067. Curran Associates, Inc., 2022.

[20] Yi-Lun Liao, Brandon Wood, Abhishek Das, and Tess Smidt. Equiformerv2: Improved equivariant transformer for scaling to higher-degree representations. *arXiv preprint arXiv:2306.12059*, 2023.

[21] Lowik Chanussot et al. Open catalyst 2020 (oc20) dataset and community challenges. *ACS Catalysis*, 2021.

[22] Richard Tran et al. The open catalyst 2022 (oc22) dataset and challenges for oxide electrocatalysts. *ACS Catalysis*, 2023.

[23] Subrahmanya Sahoo et al. Open catalyst 2025 (oc25). 2025.

[24] Benjamin Levine et al. Open molecules 2025 (omol). 2025.

[25] Anuroop Sriram et al. Opendac 2025 dataset. 2025.

[26] Luis Barroso-Luque et al. Open materials 2024 (omat24). 2024.

[27] Brandon Wood et al. Uma: Universal machine-learned atomistic potentials. 2025.

[28] Ilyes Batatia et al. A foundation model for atomistic materials chemistry. *arXiv preprint*, 2024.

[29] Jonathan Neumann et al. Orb: Fast and scalable neural network potentials. *arXiv preprint*, 2024.

[30] Janosh Riebesell, Rhys EA Goodall, Philipp Benner, Yuan Chiang, Bowen Deng, Gerbrand Ceder, Mark Asta, Alpha A Lee, Anubhav Jain, and Kristin A Persson. A framework to evaluate machine learning crystal stability predictions. *Nature Machine Intelligence*, 7(6):836–847, 2025.

[31] Kamal Choudhary, Daniel Wines, Kangming Li, Kevin F Garrity, Vishu Gupta, Aldo H Romero, Jaron T Krogel, Kayahan Saritas, Addis Fuhr, Panchapakesan Ganesh, et al. Jarvis-leaderboard: a large scale benchmark of materials design methods. *npj Computational Materials*, 10(1):93, 2024.

[32] Simon Batzner, Albert Musaelian, Lixin Sun, Mario Geiger, Jonathan P Mailoa, Mordechai Kornbluth, Nicola Molinari, Tess E Smidt, and Boris Kozinsky. E(3)-equivariant graph neural networks for data-efficient and accurate interatomic potentials. volume 13, page 2453, 2022.

[33] Yutack Park, Jaesun Kim, Seungwoo Hwang, and Seungwoo Han. Scalable parallel algorithm for graph neural network interatomic potentials in molecular dynamics simulations. *Journal of Chemical Theory and Computation*, 20(11):4857–4868, 2024.

[34] Ilyes Batatia, Dávid P Kovács, Gregor Simm, Christoph Ortner, and Gábor Csányi. MACE: Higher order equivariant message passing neural networks for fast and accurate force fields. 2022.

[35] Bowen Deng, Peichen Zhong, KyuJung Jun, Janosh Riebesell, Kevin Han, Christopher J Bartel, and Gerbrand Ceder. Chgnet as a pretrained universal neural network potential for charge-informed atomistic modelling. *Nature Machine Intelligence*, 5:1031–1041, 2023.

[36] Sheng Gong, Keqiang Yan, Tian Xie, Yang Shao-Horn, Rafael Gomez-Bombarelli, Shuiwang Ji, and Jeffrey C Grossman. Examining graph neural networks for crystal structures: limitations and opportunities for capturing periodicity. *Science Advances*, 9(45):eadi3245, 2023.

[37] Kenta Oono and Taiji Suzuki. Graph neural networks exponentially lose expressive power for node classification. *arXiv preprint arXiv:1905.10947*, 2019.

[38] Uri Alon and Eran Yahav. On the bottleneck of graph neural networks and its practical implications. *arXiv preprint arXiv:2006.05205*, 2020.

[39] Qimai Li, Zhichao Han, and Xiao-Ming Wu. Deeper insights into graph convolutional networks for semi-supervised learning. In *Proceedings of the AAAI conference on artificial intelligence*, volume 32, 2018.

[40] Weihua Hu, Bowen Liu, Joseph Gomes, Marinka Zitnik, Percy Liang, Vijay Pande, and Jure Leskovec. Strategies for pre-training graph neural networks. *arXiv preprint arXiv:1905.12265*, 2019.

[41] Jun Xia, Chengshuai Zhao, Bozhen Hu, Zhangyang Gao, Cheng Tan, Yue Liu, Siyuan Li, and Stan Z Li. Mole-bert: Rethinking pre-training graph neural networks for molecules. In *The Eleventh International Conference on Learning Representations*, 2023.

[42] Rishikesh Magar, Yuyang Wang, and Amir Barati Farimani. Crystal twins: self-supervised learning for crystalline material property prediction. *npj Computational Materials*, 8(1):231, 2022.

[43] Cheng Zheng, Bo Zong, Wei Cheng, Dongjin Song, Jingchao Ni, Wenchao Yu, Haifeng Chen, and Wei Wang. Robust graph representation

learning via neural sparsification. In *International conference on machine learning*, pages 11458–11468. PMLR, 2020.

[44] Yu Rong, Wenbing Huang, Tingyang Xu, and Junzhou Huang. Dropedge: Towards deep graph convolutional networks on node classification. *arXiv preprint arXiv:1907.10903*, 2019.

[45] Will Hamilton, Zhitao Ying, and Jure Leskovec. Inductive representation learning on large graphs. *Advances in neural information processing systems*, 30, 2017.

[46] Xianfeng Tang, Yandong Li, Yiwei Sun, Huaxiu Yao, Prasenjit Mitra, and Suhang Wang. Robust graph neural network against poisoning attacks via transfer learning. *arXiv preprint arXiv:1908.07558*, 2019.

[47] Johannes Gasteiger, Stefan Weißenberger, and Stephan Günnemann. Diffusion improves graph learning. *Advances in neural information processing systems*, 32, 2019.

[48] Kedar Karhadkar, Pradeep Kr Banerjee, and Guido Montúfar. Fosr: First-order spectral rewiring for addressing oversquashing in gnns. *arXiv preprint arXiv:2210.11790*, 2022.

[49] Jeongwhan Choi, Sumin Park, Hyowon Wi, Sung-Bae Cho, and Noseong Park. Panda: Expanded width-aware message passing beyond rewiring. *arXiv preprint arXiv:2406.03671*, 2024.

[50] Youn-Yeol Yu, Jeongwhan Choi, Jaehyeon Park, Kookjin Lee, and Noseong Park. Piorf: Physics-informed ollivier-ricci flow for long-range interactions in mesh graph neural networks. *arXiv preprint arXiv:2504.04052*, 2025.

[51] Egor Rumiantsev, Marcel F Langer, Tulga-Erdene Sodjargal, Michele Ceriotti, and Philip Loche. Learning long-range representations with equivariant messages. *arXiv preprint arXiv:2507.19382*, 2025.

[52] Kevin Han, Haolin Cong, Bowen Deng, and Amir Barati Farimani. Smooth dynamic cutoffs for machine learning interatomic potentials. *arXiv preprint arXiv:2601.21147*, 2026.

[53] Yoshua Bengio, Nicholas Léonard, and Aaron Courville. Estimating or propagating gradients through stochastic neurons for conditional computation. *arXiv preprint arXiv:1308.3432*, 2013.

[54] Luca Franceschi, Mathias Niepert, Massimiliano Pontil, and Xiao He. Learning discrete structures for graph neural networks. In *International conference on machine learning*, pages 1972–1982. PMLR, 2019.

[55] Minjie Wang, Da Zheng, Zihao Ye, Quan Gan, Mufei Li, Xiang Song, Jinjing Zhou, Chao Ma, Lingfan Yu, Yu Gai, et al. Deep graph library: A graph-centric, highly-performant package for graph neural networks. *arXiv preprint arXiv:1909.01315*, 2019.

[56] Hengrui Zhang, Zhongming Yu, Guohao Dai, Guyue Huang, Yufei Ding, Yuan Xie, and Yu Wang. Understanding gnn computational graph: A coordinated computation, io, and memory perspective. *Proceedings of Machine Learning and Systems*, 4:467–484, 2022.

[57] Kamal Choudhary, Kevin F Garrity, Andrew CE Reid, Brian DeCost, Adam J Biacchi, Angela R Hight Walker, Zachary Trautt, Jason Hattrick-Simpers, A Gilad Kusne, Andrea Centrone, et al. The joint automated repository for various integrated simulations (jarvis) for data-driven materials design. *npj computational materials*, 6(1):173, 2020.

[58] Anubhav Jain, Shyue Ping Ong, Geoffroy Hautier, Wei Chen, William Davidson Richards, Stephen Dacek, Shreyas Cholia, Dan Gunter, David Skinner, Gerbrand Ceder, et al. Commentary: The materials project: A materials genome approach to accelerating materials innovation. *APL materials*, 1(1), 2013.

[59] Deli Chen, Yankai Lin, Wei Li, Peng Li, Jie Zhou, and Xu Sun. Measuring and relieving the over-smoothing problem for graph neural networks from the topological view. In *Proceedings of the AAAI conference on artificial intelligence*, volume 34, pages 3438–3445, 2020.

[60] Kaicheng Zhang, Piero Deidda, Desmond Higham, and Francesco Tudisco. Are we measuring oversmoothing in graph neural networks correctly? In *International Conference on Learning Representations*, volume 2026, pages 143581–143609, 2026.

[61] Kai Guo, Xiaofeng Cao, Zhining Liu, and Yi Chang. Taming over-smoothing representation on heterophilic graphs. *Information Sciences*, 647:119463, 2023.

[62] Jake Topping, Francesco Di Giovanni, Benjamin Paul Chamberlain, Xiaowen Dong, and Michael M Bronstein. Understanding over-squashing and bottlenecks on graphs via curvature. *arXiv preprint arXiv:2111.14522*, 2021.

[63] Ben Finkelshtein, Xingyue Huang, Michael Bronstein, and Ismail Ilkan Ceylan. Cooperative graph neural networks. *arXiv preprint arXiv:2310.01267*, 2023.

[64] Xavier Bresson and Thomas Laurent. Residual gated graph convnets. *arXiv preprint arXiv:1711.07553*, 2017.

[65] Petar Veličković, Guillem Cucurull, Arantxa Casanova, Adriana Romero, Pietro Lio, and Yoshua Bengio. Graph attention networks. *arXiv preprint arXiv:1710.10903*, 2017.